Certified Classical and Quantum Linear Solves inside a Production Interior-Point Method: Equilibration, Low-Precision Refinement, and Conditions for a Quantum Advantage

Authors*

October 2026

Abstract

Every iteration of a primal–dual interior-point method (IPM) solves a symmetric indefinite Karush–Kuhn–Tucker (KKT) system whose conditioning deteriorates as the barrier parameter vanishes. We develop a certified inexact-solve framework for the KKT callback of the production solver CVXOPT, in which classical Krylov and simulated quantum linear solvers are subject to one acceptance test, one fallback protocol and one strict accounting, and we combine it with four methodological components: per-iteration symmetric equilibration of the augmented KKT matrix, a combined block-forcing and backward-error test, a latency-optimised GPU MINRES kernel, and a quantum solver that runs a minimax-optimal inversion polynomial at fixed low precision inside classical iterative refinement, optionally on a positive definite null-space target. We prove that the refined quantum solver contracts the residual by $\xi \le \varepsilon + (1+\varepsilon)\hat{\kappa}\zeta$ per round, bound its block-encoding queries, and show that its circuit depth is independent of the final accuracy, that equilibration shortens it by at least $\kappa(\tilde{K})/\hat{\kappa}$, and that an advantage over MINRES per Newton direction requires dense, well-equilibrated systems and is bounded by the ratio k/R of Krylov iterations to refinement rounds. Empirically, equilibration lowers the KKT condition number by a median 4.3×10^4 on `sparse_banded` and makes unpreconditioned MINRES fallback-free on every synthetic run up to dimension 79 360 and up to 9.4× faster than sparse LDL$^\mathsf{T}$ under high fill-in, whereas the factorisation remains fastest on 40 of 48 Maros–Mészáros QPs. In simulation, the quantum components reduce block-encoding queries per solve by 237×, circuit depth by 2.70× (a further 4.0× on the null-space target) and samples per solve from 3.9×10^{17} to 7.3×10^3, and the IPM converges on quantum-produced directions alone on 8/8 instances with ideal readout and on 39/40 seeded finite-sampling runs. All quantum results are classical simulations of ideal circuits; no end-to-end quantum speed-up is demonstrated.

Keywords: interior-point methods; KKT systems; inexact Newton directions; equilibration; Krylov methods; GPU computing; quantum linear-system algorithms; iterative refinement; block encoding.

1 Introduction

Interior-point methods solve convex quadratic programs (QPs) in a few dozen Newton iterations, and each iteration requires the solution of a KKT linear system. Production solvers factorise this system—by a sparse LDL$^\mathsf{T}$ decomposition of the augmented matrix or a Cholesky decomposition of a reduced one—and the factorisation dominates the run time whenever its fill-in is severe. Replacing

*Author names and affiliations to be inserted. Correspondence: `ldrresearch2025@gmail.com`.

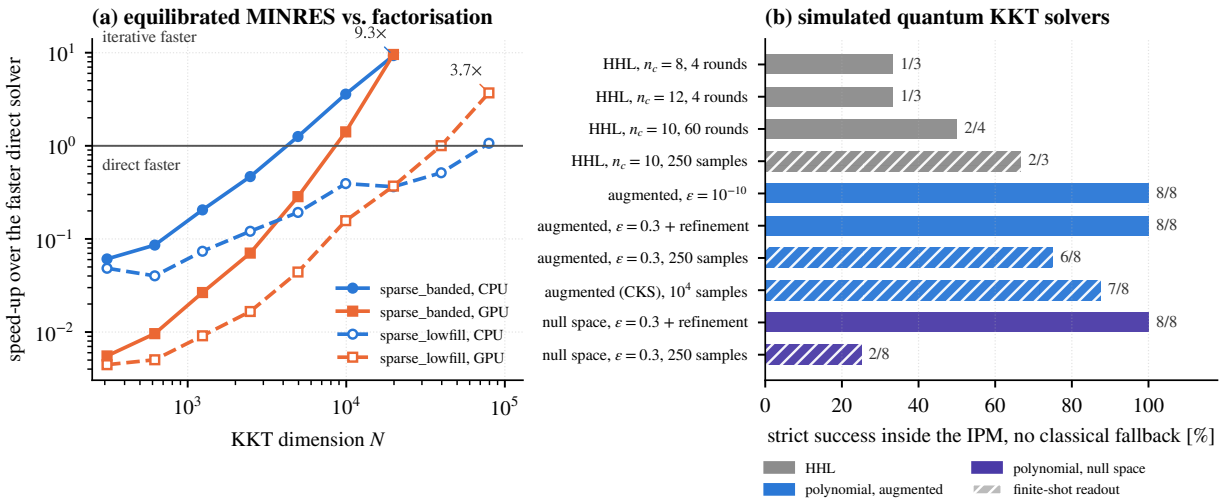


Fig. 1: Principal outcomes. (a) Speed-up of equilibrated MINRES over the faster of the two direct solvers (ratio of median whole-run times) versus KKT dimension, on `sparse_banded` (high Cholesky fill-in, solid) and on the control family `sparse_lowfill` (low fill-in, dashed), on the CPU and on a Tesla T4 GPU. (b) Share of IPM runs in which a simulated quantum KKT solver drove the unmodified solver to the optimum with every direction accepted and no classical fallback.

it by an iterative Krylov method is a classical idea [9, 28], and the convergence theory of inexact IPMs prescribes how accurate each direction must be [2, 6, 20, 29]. The same step has more recently been proposed as the point of insertion for quantum linear-system algorithms inside quantum IPMs [5, 35, 42, 43, 62], whose end-to-end practicality has been questioned by resource analyses [19].

Whether an inexact or a quantum KKT solver is useful inside an IPM depends on three questions: how accurate a Newton direction must be for the outer iteration to converge; how that accuracy can be certified at a cost below that of the solve itself; and which solvers can deliver it on the matrices that an IPM actually produces. Existing studies address these questions separately. Inexact-IPM analyses fix the forcing condition but are typically evaluated with purpose-built IPMs and preconditioners; quantum-IPM demonstrations simulate phase-estimation HHL on linear systems of dimension at most 16 and reach a precision of about 10^{-4} [43]; and complexity bounds for quantum IPMs are stated in terms of a condition number that grows without bound along the central path. No study has subjected classical and quantum solvers to the same certification inside an unmodified production IPM, and none has quantified which methodological choices reduce the resources a quantum solver needs, by how much, and how far the reduced cost remains from a quantum advantage.

This paper addresses these questions within a single framework. We use the `kktsolver` callback of CVXOPT `coneqp` [3, 4], which allows any solver to replace the KKT factorisation while the rest of the algorithm—Nesterov–Todd scaling, the predictor–corrector scheme, step lengths and stopping tests—remains untouched. Every solver, classical or quantum, is subject to the same acceptance test, the same classical-fallback protocol and the same accounting, so that a reported success means that the IPM converged on directions produced by the solver under test alone. Fig. 1 summarises the principal outcomes.

Methodological contributions. The contributions are the following.

C1. *A certified inexact-solve framework for a production IPM* (Section 4). A combined acceptance

test—per-block forcing $\eta = 0.1$ together with a normwise backward error $\omega \leq 10^{-9}$—certifies every direction at 22–34 % more Krylov work than the cheapest single test that is also fallback-free, and a three-level accounting (accepted, fallback-free, strict) separates the contribution of the solver under test from that of the safeguards. The framework applies unchanged to classical Krylov methods, noisy exact solves, HHL and polynomial quantum solvers.

C2. *Per-iteration symmetric equilibration of the augmented KKT matrix* (Sections 4.2 and 7.1). Ruiz equilibration applied to every new KKT matrix, rather than once to the problem data, lowers the condition number by a median 4.3×10^4 on `sparse_banded` and 5.4×10^3 on `dense_kappa`. It makes unpreconditioned MINRES fallback-free on all synthetic runs, whereas every unscaled variant needs fallback in 38–50 % of them. For the quantum solver, Corollary 5.5 shows that the same transformation shortens the circuit by at least the factor by which it reduces the condition number (1.2×10^5 and 1.2×10^4 at $\varepsilon = 0.3$).

C3. *A measured precision law for Newton directions* (Section 7.6). At every IPM tolerance τ from 10^{-8} to 10^{-4}, the largest relative direction error tolerated on every instance lies between $\tau/10$ and τ. This fixes the accuracy target of any inexact or quantum solver and, combined with Corollary 5.4, implies that refinement needs only $\mathcal{O}(\log(1/\tau))$ rounds.

C4. *A latency-optimised GPU MINRES kernel and a controlled scaling study* (Sections 4.3, 7.3 and 7.4). A kernel with one host synchronisation per iteration, fused vector updates and adaptively spaced convergence checks is 1.14–1.29× faster per iteration than a direct CuPy transcription. Equilibrated MINRES overtakes the sparse LDL^T factorisation at $N \approx 5 \times 10^3$ under high fill-in and is 9.4× faster at $N = 19\,840$; the GPU pays off from $N \approx 2 \times 10^4$ (3.75× at 79 360). A low-fill control family and 48 Maros–Mészáros QPs delimit the regime in which these gains hold.

C5. *A low-precision refined polynomial quantum solver* (Sections 4.4.3–4.4.6 and 8.3–8.6). The minimax-optimal inversion polynomial of Sünderhauf et al. [55], executed at fixed precision $\varepsilon = 0.3$ inside classical iterative refinement with a few hundred samples per round, needs 2.70× fewer walk steps than the Childs–Kothari–Somma expansion and 237× fewer block-encoding queries per solve than a high-precision circuit. It drives the unmodified IPM to the optimum with every direction quantum-produced on 8/8 instances with ideal readout and on 39/40 seeded runs at 4 000 samples per round. We identify the mechanism of the remaining failures: directions accepted close to the forcing threshold lengthen the IPM run into regions of larger condition number.

C6. *A positive definite null-space target with a closed-form polynomial* (Sections 4.4.4 and 8.7). The orthogonal-subspace system of inexact-feasible quantum IPMs is positive definite, so a shifted Chebyshev polynomial with closed-form coefficients and degree $\mathcal{O}(\sqrt{\kappa}\log(\kappa/\varepsilon))$ suffices. Inside the IPM it needs 4.0× fewer walk steps than the augmented target; Corollary 5.6 gives the criterion under which it is the cheaper target and explains why its advantage vanishes late in the run.

C7. *A complexity analysis with explicit conditions for a quantum advantage* (Section 5). Theorem 5.2 bounds the contraction, the number of rounds and the block-encoding queries of the refined solver; three corollaries quantify the effect of precision, equilibration and the choice of target; and Proposition 5.7 states when the hybrid scheme can be cheaper than classical MINRES per Newton direction. The analysis shows that an advantage is possible only for dense, well-equilibrated systems, that it is polynomial and bounded by the ratio k/R of Krylov iterations to refinement rounds, and it locates our measurements relative to that boundary (Fig. 15).

Significance. On the classical side, the results show that equilibration alone, without a preconditioner, makes a Krylov solver a certified and competitive replacement for the factorisation whenever fill-in is the bottleneck and the problem is regular, and that latency rather than arithmetic decides where a GPU kernel pays off. On the quantum side, the three dominant cost drivers of a QLSA inside an IPM—the condition number, the required precision and the readout of a classical vector—can each be reduced by orders of magnitude through classical pre- and post-processing that is cheap relative to the solve: equilibration acts on the condition number, low-precision refinement removes the $1/\varepsilon_f^2$ sample dependence of direct readout and decouples the circuit depth from the IPM tolerance, and the choice of polynomial and target lowers the constants. These reductions apply to any QLSA-based IPM, and Proposition 5.7 quantifies the gap that remains. All quantum results are classical simulations of ideal circuits with oracle access to the block encoding and state preparation; they establish algorithmic feasibility and relative resource requirements, not a speed-up over classical solvers, and at the simulated sizes the absolute query counts exceed the classical cost by many orders of magnitude.

Organisation. Section 2 reviews the related literature. Section 3 introduces the KKT system of `coneqp` and its conditioning. Section 4 presents the methodology: the certified solve protocol, equilibration, the MINRES kernel and the quantum solvers. Section 5 contains the complexity analysis. Section 6 describes the experimental setup, and Sections 7 and 8 report the classical and quantum results. Section 9 discusses significance, the outlook for a quantum advantage and the limitations, and Section 10 concludes. The appendices document reproducibility, the SDPLIB results and the replication across six computing sessions.

2 Related work

2.1 Inexact interior-point methods and the conditioning of KKT systems

Bellavia [6], Al-Jeiroudi and Gondzio [2] and Gondzio [29] establish forcing conditions of the form $\|r\| \le \eta\mu$ under which an inexact IPM converges; Gondzio's analysis for convex QP requires the residual bound only in the complementarity block, which is related to our per-block test. Zanetti and Gondzio [63] derive IPM-specific termination criteria for CG and MINRES that estimate the IPM indicators inside the Krylov loop; they require access to the current iterate, which the `kktsolver` interface of `coneqp` does not provide, but our measurement of over-solving (Section 7.2) quantifies what such a criterion could save. The boundedness of the unreduced (or suitably scaled) KKT matrix as $\mu \to 0$ under strict complementarity and linear independence, in contrast to the $\mathcal{O}(\mu^{-2})$ growth of reduced and normal-equation systems, is established in [31, 44], and the benign structure of the ill-conditioning in [22, 61]; the general saddle-point background is surveyed in [8]. In this terminology, the KKT matrix of CVXOPT with its $-I$ block is the augmented form, and our contribution is the measured condition-number trajectory after symmetric equilibration and its consequences for classical and quantum solvers. Preconditioners for IPM sequences [7, 9, 30, 50] and randomised Nyström preconditioning [16, 23] are complementary; Section 7.7 shows that the low-rank premise of the latter does not hold on our families.

2.2 Scaling, preconditioning and Krylov kernels on GPUs

Ruiz equilibration [36, 52] is applied by OSQP [54] once to the problem data; we apply it to the barrier KKT matrix at every IPM iteration. That Krylov methods on GPUs are bounded by synchronisation latency is known from the pipelined-CG literature [25]; recent GPU IPM

solvers [14, 47, 51] prefer direct factorisation on the device, and first-order GPU solvers [38, 53] avoid the KKT solve altogether. Mixed-precision iterative refinement [13, 60] motivates our test of single-precision inner solves. For benchmarking we follow the performance-profile methodology of Dolan and Moré [21] and use the Maros–Mészáros collection [40], the standard real-world test set for convex QP solvers [54]. Recycled Krylov methods for sequences of systems [49, 59] and learned deflation spaces [39] address the growth of the iteration count that we measure in Section 7.2; learned surrogates of the IPM linear solve [24] reach residuals of about 10^{-2} and serve as warm starts rather than replacements.

2.3 Quantum linear-system algorithms in optimisation

HHL [34], its polynomial successors based on linear combinations of unitaries and quantum signal processing [15, 26, 37, 41, 57], and solvers with optimal $\mathcal{O}(\kappa \log(1/\varepsilon))$ query complexity [17, 18] provide the quantum linear-algebra primitives; block-encoding constructions are surveyed in [11, 32], and the textbook treatment of quantum optimisation in [45]. Aaronson's caveats [1] and the tomography lower bounds of van Apeldoorn et al. [58] explain why extracting a classical solution costs at least $\Omega(N/\varepsilon)$ state preparations, and dequantisation results [56] delimit the regime in which low-rank structure removes any advantage. Quantum IPMs [5, 35, 42, 43, 62] use iterative refinement to escape the $1/\varepsilon$ dependence of tomography; the standalone analysis of that scheme, with simulator and hardware experiments on systems of dimension 8 and 64, is due to Harkness et al. [33], and the resource estimate for an end-to-end quantum IPM to Dalzell et al. [19]. Inexact-feasible quantum IPMs [5, 62] solve the Newton system in a basis of the null space of the equality constraints (the orthogonal-subspace system), which preserves primal feasibility under inexact solves and yields a positive definite target. The VQE-based cutting-plane study of Ozbaygin, Kocuk and Moran [46] for SDP identifies power-of-two padding, Pauli density and tomography as practical obstacles to a variational separation oracle, all three of which we quantify for KKT matrices.

2.4 Positioning

The present study differs from the quantum-IPM demonstrations above in four respects: the IPM is an unmodified production solver rather than a purpose-built one; every quantum direction is subject to the same acceptance test and strict accounting as the classical ones, so that a success means that the IPM converged on quantum directions alone; the quantum solver is a walk-level polynomial solver operated at low precision inside refinement rather than phase-estimation HHL; and the resource reductions of each methodological component are both measured and derived in closed form, together with the conditions under which they could amount to an advantage. It differs from the inexact-IPM literature in that the classical Krylov results rest on equilibration alone, without a preconditioner, and are reported together with a low-fill control family and a real-world QP collection on which the direct solver wins. To our knowledge, the measured condition-number trajectory of the equilibrated augmented matrix, the measured precision law of a Newton direction, and the comparison of HHL and polynomial solvers under one protocol inside a production IPM have not been reported before.

3 Problem setting

3.1 The KKT system of `coneqp`

We consider the convex QP

$$\min_{x\in\mathbb{R}^n} \ \tfrac{1}{2}x^{\mathsf{T}}Px + q^{\mathsf{T}}x \quad \text{s.t.} \quad Gx + s = h,\ Ax = b,\ s \geq 0, \tag{1}$$

with $P \succeq 0$ (in all synthetic instances $P \succ 0$), $G \in \mathbb{R}^{m\times n}$ and $A \in \mathbb{R}^{p\times n}$ of full row rank. `coneqp` is a primal–dual path-following method with Nesterov–Todd (NT) scaling; for the nonnegative orthant the scaling matrix is diagonal, $W = \operatorname{diag}(w)$ with $w_i = \sqrt{s_i/z_i}$, where z is the dual variable associated with s. Each iteration factorises the KKT matrix once and solves with it twice (affine and combined directions), and one further solve initialises the method. The solver calls a user function `kktsolver(W)` that must return a routine solving

$$\tilde{K}(W)\,u = r, \qquad \tilde{K}(W) = \begin{bmatrix} P & A^{\mathsf{T}} & G^{\mathsf{T}}W^{-1} \\ A & 0 & 0 \\ W^{-1}G & 0 & -I \end{bmatrix}, \qquad u = \begin{bmatrix} u_x \\ u_y \\ u_z \end{bmatrix}, \tag{2}$$

where the right-hand side is assembled from the solver's residuals and the returned z-component is Wu_z; we verified this contract against CVXOPT's own `kkt_ldl` routine to 2×10^{-16}. The matrix $\tilde{K}$ is symmetric indefinite of dimension $N = n + p + m$ with inertia $(n,\, p + m)$ when $P \succ 0$.

3.2 Target systems and their conditioning

Two reductions of (2) are standard. Eliminating u_z yields the quasi-definite system

$$\begin{bmatrix} S & A^{\mathsf{T}} \\ A & 0 \end{bmatrix} \begin{bmatrix} u_x \\ u_y \end{bmatrix} = \begin{bmatrix} r_x + \hat{G}^{\mathsf{T}} r_z \\ r_y \end{bmatrix}, \qquad S = P + \hat{G}^{\mathsf{T}}\hat{G}, \quad \hat{G} = W^{-1}G, \tag{3}$$

and eliminating u_y as well yields the normal equations with Schur complement $AS^{-1}A^{\mathsf{T}}$. As the IPM converges, $w_i \to 0$ on the active constraints and $w_i \to \infty$ on the inactive ones, so that $\hat{G}^{\mathsf{T}}\hat{G}$ contains entries of order μ^{-1} and $\kappa(S) = \mathcal{O}(\mu^{-2})$. The following statement specialises results of Greif, Moulding and Orban [31] and Morini, Simoncini and Tani [44] to (2); it motivates the choice of target system.

Proposition 3.1. *Let the QP* (1) *satisfy strict complementarity at its solution and let* $[A^{\mathsf{T}}\ G_{\mathcal{A}}^{\mathsf{T}}]$ *have full column rank, where* $G_{\mathcal{A}}$ *collects the active rows of* G*. Along a sequence of IPM iterates converging to the solution, the reduced matrix in* (3) *has* $\kappa = \mathcal{O}(\mu^{-2})$*, whereas after symmetric diagonal scaling of the inactive rows and columns of the* z*-block the augmented matrix* (2) *converges to a nonsingular limit and its condition number is bounded uniformly in* μ*.*

A third target system keeps the equality constraints exactly. Let $Z \in \mathbb{R}^{n\times(n-p)}$ have orthonormal columns spanning $\operatorname{null}(A)$ and let $Ax_p = r_y$. Writing $u_x = x_p + Zv$ reduces (3) to the positive definite system

$$\big(Z^{\mathsf{T}}SZ\big)\,v = Z^{\mathsf{T}}\big(r_x + \hat{G}^{\mathsf{T}}r_z - Sx_p\big), \qquad u_z = \hat{G}u_x - r_z, \qquad u_y = (AA^{\mathsf{T}})^{-1}A\big(r_x - Pu_x - \hat{G}^{\mathsf{T}}u_z\big), \tag{4}$$

which is the orthogonal-subspace system of inexact-feasible quantum IPMs [5, 62]. Its dimension is $n - p$ instead of N and it is definite, so CG applies classically and a Chebyshev polynomial of degree $\mathcal{O}(\sqrt{\kappa}\log(\kappa/\varepsilon))$ suffices on a quantum computer (Section 4.4.4). Two properties count against it: $Z^{\mathsf{T}}SZ$ inherits the $\mathcal{O}(\mu^{-2})$ growth of S under the hypotheses of Proposition 3.1, and the multiplier u_y is recovered through P, so that an error δ in u_x becomes an error of up to $\|P\|\,\|\delta\|\,/\sigma_{\min}(A)$ in u_y. Section 8.7 measures both effects.

4 Methodology

This section presents the components of the framework. Section 4.1 defines the acceptance tests and the solve protocol that every solver follows; Section 4.2 the per-iteration equilibration; Section 4.3 the MINRES solver and the latency model of its GPU kernel; and Section 4.4 the quantum solvers, their readout and the refinement scheme.

4.1 Certified inexact solves

Let u be a candidate solution of (2) with residual $e = \tilde{K}u - r$, partitioned into the x, y and z blocks. All tests are evaluated on the original, unscaled system. A relative-residual or backward-error test with tolerance τ_ρ or τ_ω accepts u when $\rho(u) \le 10\tau_\rho$ or $\omega(u) \le 10\tau_\omega$; the factor of ten is the same for every method, and tolerances are quoted as τ_ρ and τ_ω throughout.

Relative residual. $\rho(u) = \|e\| \,/\, \|r\|$ is the most common stopping test. It is a forward-error test in disguise—$\|r\|$ can be small while $\left\|\tilde{K}\right\| \|u\|$ is not—and becomes unattainable late in the IPM run (Section 7.1).

Normwise backward error. $\omega(u) = \|e\| \,/\, \big(\left\|\tilde{K}\right\|_2 \|u\| + \|r\| \big)$, with $\left\|\tilde{K}\right\|_2$ estimated by 20 power iterations, is the smallest relative perturbation of the data for which u is an exact solution.

Lemma 4.1 (forward error from backward error)**.** *If $\omega(u) \le \omega$ and $\omega\kappa(\tilde{K}) < 1$, then $\|u - u^\star\| \,/\, \|u^\star\| \le 2\omega\kappa(\tilde{K})/(1 - \omega\kappa(\tilde{K}))$, where $u^\star = \tilde{K}^{-1}r$.*

Proof. By the Rigal–Gaches theorem, u solves $(\tilde{K}+\Delta)u = r+\delta$ with $\|\Delta\| \le \omega \left\|\tilde{K}\right\|$ and $\|\delta\| \le \omega \|r\|$; the standard perturbation bound for linear systems gives the claim. □

Lemma 4.1 explains why a backward-error test alone can let the IPM fail when $\kappa(\tilde{K})$ is large: a tolerance of 10^{-8} on a system with $\kappa = 10^{10}$ certifies nothing about the forward error.

Block forcing. The per-block ratio

$$\beta(u) = \max_{b \in \{x,y,z\}} \frac{\|e_b\|}{\max(\|r_b\|, \phi_b)}, \qquad \phi_b = 10^{-9} \max(1, \|\mathrm{data}_b\|),$$

with the absolute floor ϕ_b, is the form used in inexact-IPM convergence proofs [6, 29]: it bounds the residual of each block relative to that block's right-hand side. Because $r_y \to 0$ once the iterates are primal feasible, the equality block turns into an absolute test at level $\eta\phi_y$ and becomes the binding block in the late iterations (Section 7.2).

Combined test. We accept a direction when

$$\beta(u) \le \eta \quad \text{and} \quad \omega(u) \le 10\tau_\omega, \tag{5}$$

with $\eta = 0.1$, and $\tau_\omega = 10^{-10}$ for the classical Krylov methods and 10^{-9} for the quantum solvers, i.e. $\omega \le 10^{-9}$ and $\omega \le 10^{-8}$. The block test carries the IPM-specific structure and the backward-error test the certificate of Lemma 4.1.

Algorithm 1: Certified KKT solve (one call of the `kktsolver` routine).

Input: $\tilde{K}$, r, base solver $\mathcal{S}$, acceptance test $c \in \{\rho, \omega, \beta, \text{comb}\}$ with threshold, refinement budget $j_{\max}$, flag `fallback`

Output: direction (u_x, u_y, Wu_z) and a log entry

$u \leftarrow \mathcal{S}(\tilde{K}, r)$;

`// Krylov, exact, noisy exact, HHL or polynomial quantum solver`

for $j = 1, \ldots, j_{\max}$ ***while*** $c(u)$ *fails* **do**

  $\delta \leftarrow \mathcal{S}(\tilde{K},\, r - \tilde{K}u)$; $u \leftarrow u + \delta$;

  `// iterative refinement with the same solver`

if $c(u)$ *fails* ***and*** `fallback` **then**

  $u \leftarrow \tilde{K}^{-1} r$ by a direct factorisation; count a fallback;

if $c(u)$ *fails* ***and not*** `fallback` **then**

  count an *unaccepted direction used*; pass u to the IPM;

log $\rho(u), \omega(u), \beta(u)$, Krylov iterations or quantum resources, and timings;

Solve protocol and accounting. Every solver other than the unmodified baseline follows Algorithm 1. The accounting distinguishes three outcomes of a complete IPM run: *accepted*—the final iterate satisfies independent tolerances (primal and dual residuals $\leq 10^{-6}$, relative gap $\leq 10^{-5}$, cone violation $\leq 10^{-8}$, objective error $\leq 10^{-5}$ relative to the known optimum, or to the objective of the unmodified solver on the Maros–Mészáros instances); *fallback-free*—accepted without any classical fallback; and *strict*—fallback-free, and no direction that failed its acceptance test was ever passed to the IPM. Only a strict run counts as a success of the solver under test. We also report CVXOPT's own status, which is *optimal* only at its tighter default tolerances (10^{-7}). The two can differ for the simulated quantum solvers: when a circuit exceeds the simulation's depth budget, the solve returns a non-finite direction and `coneqp` stops with status *unknown*, possibly after the iterate has already met the independent tolerances; such runs count as strict and are marked separately. Runs are never terminated because of a large residual; only non-finite directions abort a run, as in CVXOPT itself.

The fallback factorisation is either the LU decomposition of $\tilde{K}$ or the CHOLMOD factorisation of the reduced system (3), depending on the session (Table 3); on the synthetic families no fallback occurs for the equilibrated methods, so the choice matters only for the unscaled ablations and the real-world QPs.

4.2 Per-iteration symmetric equilibration

The scaling required by Proposition 3.1 depends on the unknown active set. Ruiz equilibration [36, 52] is a data-independent substitute: with $d \in \mathbb{R}^N_{>0}$ and $D = \operatorname{diag}(d)$, iterate

$$d_i \leftarrow d_i \Big/ \sqrt{\max_j |d_i \tilde{K}_{ij} d_j|}, \qquad i = 1, \ldots, N, \tag{6}$$

for a fixed number of sweeps (six in all experiments), which drives every row and column of $D\tilde{K}D$ towards unit ∞-norm. Because the ill-conditioning of $\tilde{K}$ is diagonal in nature—the z-rows scale like w_i^{-1}—the iteration (6) recovers most of the effect of the scaling in Proposition 3.1: Section 7.2 measures $\kappa(D\tilde{K}D) \in [80, 180]$ while $\max w / \min w$ grows to 1.4×10^6, and Table 15 reports the reduction across all families. Equilibration costs $\mathcal{O}(\operatorname{nnz}(\tilde{K}))$ per sweep on the CSR arrays and is

applied to every new $\tilde{K}(W)$. A solve of $D\tilde{K}D\,\hat{u} = Dr$ returns $u = D\hat{u}$ exactly; the acceptance test is nevertheless evaluated on the original system. The same scaled matrix is the input of the quantum solvers, for which its condition number $\hat{\kappa} := \kappa(D\tilde{K}D)$ determines the circuit depth (Section 5).

4.3 Equilibrated MINRES and a latency model for GPU kernels

MINRES [48] applied to $D\tilde{K}D\,\hat{u} = Dr$ minimises $\left\|D(\tilde{K}u - r)\right\|$ over the Krylov space. We stop it by evaluating the acceptance test on the original system every c iterations ($c = 8$, or adaptively every $\max(8, \lfloor k/16 \rfloor)$ iterations at iteration k), so that the stopping test is the certificate. For a symmetric indefinite matrix with eigenvalues in $[-1, -a] \cup [a, 1]$ the MINRES residual bound gives $\mathcal{O}(\kappa \log(1/\varepsilon))$ iterations, twice the CG bound for a definite matrix with the same κ; this is why, even at $\kappa \approx 10^2$, the counts in Section 7.2 reach 10^3 per solve, and why the distribution of the eigenvalues around ± 1 matters more than κ itself. We also evaluated alternative targets: the reduced system (3), matrix-free and Jacobi-scaled, and the square-root system $B = [L^{\mathsf{T}}; \hat{G}]$ with $S = B^{\mathsf{T}}B$, solved by LSQR. Both lose to the augmented system late in the run because their conditioning carries W^{-2}.

Latency model. A MINRES iteration performs one sparse matrix–vector product, two inner products and five vector updates. On a GPU, the time per iteration is approximately

$$t_{\text{iter}} \approx n_{\text{sync}}\, t_{\text{sync}} + n_{\text{kern}}\, t_{\text{launch}} + \frac{\text{flops}}{\text{throughput}}, \tag{7}$$

where every scalar that the host needs (the Lanczos coefficients α and β) forces a synchronisation. A direct CuPy transcription—the *baseline kernel*—has $n_{\text{sync}} = 2$ and $n_{\text{kern}} \approx 15$ per iteration plus an explicit convergence check every 8 iterations with about 9 synchronisations; we measure 0.68–0.73 ms per iteration *independently of* N up to $N = 8 \times 10^4$, whereas the mat-vec alone costs 0.07 ms. The *latency-optimised kernel* computes α and β on the device and returns them in one transfer, fuses the two recurrences into two kernels, spaces the convergence checks adaptively, evaluates the complete acceptance test with one synchronisation, and keeps the sparsity pattern resident on the device. The same code runs on the CPU with NumPy, where it uses a CSR-level equilibration sweep.

4.4 Quantum linear solvers

4.4.1 Problem, input model and padding

The quantum linear-system problem for (2) is posed on the equilibrated matrix $\hat{K} = D\tilde{K}D/\alpha$, $\alpha = \left\|D\tilde{K}D\right\|_2$, padded with $\lambda_{\max} I$ to dimension 2^{n_d}, and the right-hand side $\hat{r} = Dr$. Padding with a value at least $\lambda_{\max}$ avoids the null-space collapse reported for zero padding in [46]. The spectrum of $\hat{K}$ lies in $\mathcal{D}_{\hat{\kappa}} := [-1, -1/\hat{\kappa}] \cup [1/\hat{\kappa}, 1]$. We assume oracle access to a $(1,1)$-block encoding $U_{\hat{K}} = \begin{bmatrix} \hat{K} & \sqrt{I - \hat{K}^2} \\ \sqrt{I - \hat{K}^2} & -\hat{K} \end{bmatrix}$, to state preparation of $|\hat{r}\rangle$, and to α and $\hat{\kappa}$. The cost of these oracles is not simulated, but the inflation of the effective condition number that a concrete construction would cause is measured: a sparse-access block encoding would multiply it by a median $4.0\times$ and a Pauli-LCU encoding by $2.8\times$ on our matrices, which use 99 % of the $N(N+1)/2$ Pauli strings admissible for a real symmetric operator—the same density that [46] reports for SDP iterates.

4.4.2 Phase-estimation HHL

Phase estimation with n_c clock qubits resolves eigenvalues to $\Delta = \lambda_{\max} 2^{-n_c}$; a controlled rotation by $\arcsin(C/\hat{\lambda})$, with $C = \lambda_{\min}$ (κ-*known* mode) or $C = \Delta$ (*resolution* mode), followed by post-selection

prepares $|\hat{K}^{-1}\hat{r}\rangle$ with a relative error of order $\kappa 2^{-n_c}$ on the components along eigenvectors whose eigenvalue is not resolved. The error is a discretisation of the spectrum and therefore systematic: averaging over shots cannot reduce it below about $\kappa 2^{-n_c}$. Our gate-level statevector simulator, validated against Qiskit with fidelity 1.0, implements a signed clock register, exact reciprocal rotations and a reference branch for sign and scale recovery.

4.4.3 Polynomial solver: qubitised walk and linear combination of unitaries

Let $\mathcal{W} = (2\Pi - I)U_{\hat{K}}$ be the qubitised walk with $\Pi = |0\rangle\langle 0| \otimes I$. Then $\Pi\mathcal{W}^k\Pi = T_k(\hat{K})$, the Chebyshev polynomial of the first kind, and any odd polynomial $p(x) = \sum_j c_j T_{2j+1}(x)$ can be applied by a linear combination of unitaries with $\mathrm{PREP}|0\rangle = \sum_j \sqrt{|c_j|/\,\|c\|_1}\,|j\rangle$ and $\mathrm{SELECT} = \sum_j |j\rangle\langle j| \otimes \mathcal{W}^{2j+1}$ [15, 26]. The post-selected output is $p(\hat{K})|\hat{r}\rangle/\,\|c\|_1$, obtained with success probability

$$p_{\mathrm{succ}} = \left\| p(\hat{K})\hat{r} \right\|^2 / \left(\|c\|_1^2 \, \|\hat{r}\|^2 \right), \tag{8}$$

and the circuit makes $d = \deg p$ queries to $U_{\hat{K}}$ (walk steps).

Inversion polynomials. We compare two polynomials. The Childs–Kothari–Somma (CKS) expansion truncates $\big(1-(1-x^2)^b\big)/x$ with $b = \lceil \kappa^2 \log(\kappa/\varepsilon) \rceil$ to degree $d = 2\lceil \sqrt{b \log(4b/\varepsilon)} \rceil + 1$ [15]. The minimax-optimal odd polynomial for $1/x$ on $\mathcal{D}_\kappa$ is given in closed form by Sünderhauf et al. [55] (their Theorem 1), with uniform error $\varepsilon_{2n-1} = (1-a)^n/(a(1+a)^{n-1})$, $a = 1/\kappa$; hence the degree

$$d = 2n - 1, \qquad n = \left\lceil \frac{\log(1/\varepsilon) + \log\kappa + \log(1+a)}{\log(1+a) - \log(1-a)} \right\rceil, \qquad d \;\sim\; \kappa \log \tfrac{\kappa}{\varepsilon}. \tag{9}$$

We evaluate it with their stable recurrence and obtain its Chebyshev coefficients by exact interpolation at $d+1$ nodes (a discrete cosine transform). Table 10 and Fig. 10 report the degrees.

4.4.4 A polynomial for the positive definite null-space target

For the null-space system (4), scaled to the spectrum $[a, 1]$ with $a = 1/\kappa$, the Chebyshev expansion of $1/x$ in the shifted variable $y = (2x - 1 - a)/(1-a) \in [-1, 1]$ has closed-form coefficients

$$\frac{1}{x} = \sum_{k \geq 0} c_k T_k(y), \qquad c_0 = \sqrt{\kappa}, \quad c_k = 2\sqrt{\kappa}\,(-q)^k, \quad q = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}. \tag{10}$$

Truncation after degree d leaves an error of at most $\sqrt{\kappa}(\sqrt{\kappa}+1)q^{d+1}$; hence $d \approx \frac{1}{2}\sqrt{\kappa}\log(\kappa/\varepsilon)$, against $d \approx \kappa\log(\kappa/\varepsilon)$ for the odd polynomial (9) of an indefinite matrix with the same κ. The 1-norm satisfies $\|c\|_1 < \sqrt{\kappa}\,(1+q)/(1-q) = \kappa$, so that $p_{\mathrm{succ}} \gtrsim \kappa^{-2}$ with the condition number of the null-space system. The walk acts on a block encoding of the shifted matrix $(2\hat{K} - (1+a)I)/(1-a)$, whose eigenvalues fill $[-1, 1]$; since $\Pi\mathcal{W}^k\Pi = T_k$ for every k, SELECT applies $\mathcal{W}^k$ for $k = 0, \ldots, d$. Constructing that block encoding from one of $\hat{K}$ by an LCU incurs a normalisation factor $(3 + a)/(1-a) \approx 3$, which would lengthen the circuit by about the same factor; like the other oracle costs, it is reported but not simulated. The completion of the direction by (4) (x_p, u_z and u_y) is classical.

4.4.5 Readout

A reference branch $|\hat{r}\rangle \mapsto (|\hat{r}\rangle \otimes |0\rangle_f + |\hat{K}\hat{r}_{\mathrm{ref}}\rangle \otimes |1\rangle_f)/\sqrt{2}$ is solved together with the system. Measuring the data register in the computational basis gives the magnitudes $|x_k|$ and, through the reference branch, the scale; measuring the flag qubit in the X basis gives the signs. We denote by S the number of accepted samples per measurement setting, so that one solve uses $2S$ samples. Amplitude amplification is emulated: accepted samples are drawn from the post-selected distribution, and each is charged $1/\sqrt{p_{\mathrm{succ}}}$ circuit repetitions.

4.4.6 Low-precision iterative refinement

Let $\mathcal{O}(\hat{K}, b, \xi)$ return a classical vector $\tilde{u}$ with $\left\|\hat{K}\tilde{u} - b\right\| \le \xi \|b\|$. Starting from $u^{(0)} = 0$ and setting $u^{(k+1)} = u^{(k)} + \mathcal{O}(\hat{K}, r^{(k)}, \xi)$ with $r^{(k)} = b - \hat{K}u^{(k)}$ gives $\left\|r^{(k)}\right\| \le \xi^k \|b\|$ [33, Thm. 4.1], and hence $\lceil \log(1/\varepsilon_f)/\log(1/\xi) \rceil$ rounds for any fixed $\xi < 1$. Inside Algorithm 1, the quantum solver plays the role of $\mathcal{S}$, and the refinement loop of the algorithm is exactly this iteration. The contraction factor ξ combines the polynomial error ε and the readout error. Empirically (Section 8.8), the latter is $c_0\sqrt{N/(2S)}$ with $c_0 = 1.10$, which leads to the cost model

$$Q(\varepsilon, S) \;\approx\; \underbrace{\frac{\log(1/\varepsilon_f)}{\log\big(1/\xi(\varepsilon,S)\big)}}_{\text{rounds}} \times \underbrace{2S}_{\text{samples}} \times \underbrace{\frac{d(\hat{\kappa},\varepsilon)}{\sqrt{p_{\mathrm{succ}}}}}_{\text{queries per sample}}, \qquad \xi(\varepsilon,S) \approx \max\Big\{\varepsilon,\; c_0\sqrt{N/(2S)}\Big\}. \tag{11}$$

Since d grows only like $\log(1/\varepsilon)$ while the number of rounds grows like $1/\log(1/\xi)$, Q is minimised at *large* ε and *small* S: the cheapest operating point in Section 8.4 is $\varepsilon = 0.3$ with a few hundred samples per round, not a high-precision circuit with many samples. Equation (11) also shows what refinement cannot repair: a solver whose error is not a contraction of the residual—$\xi \ge 1$ on some eigendirections, as for HHL when its clock does not resolve the smallest eigenvalues—does not converge at any budget. Section 5 turns (11) into a rigorous bound.

5 Complexity analysis and conditions for a quantum advantage

This section establishes the cost of the refined polynomial solver, derives the effect of each methodological component on that cost, and states the conditions under which the hybrid scheme can be cheaper than a classical Krylov solver for one Newton direction. Throughout, $\hat{K} = D\tilde{K}D/\alpha$ is the equilibrated and normalised KKT matrix of Section 4.4, $\hat{\kappa} = \kappa(D\tilde{K}D)$, and residuals are measured in the scaled variables; the acceptance test of Algorithm 1 is evaluated on the original system.

Assumption 5.1 (input and output model)**.** (i) A block encoding $U_{\hat{K}}$ is available, one query per walk step. (ii) For every classical vector $b \in \mathbb{R}^N$, a unitary preparing $|b\rangle = b/\|b\|$ is available. (iii) α and $\hat{\kappa}$, or upper bounds on them, are known. (iv) A readout procedure returns, from S accepted samples in each of two measurement settings, an estimate $\tilde{v}$ of the vector v whose normalised state the circuit prepares, rescaled by the reference branch, with $\|\tilde{v} - v\| \le \zeta \|v\|$.

Theorem 5.2 (refined low-precision polynomial solve)**.** *Let Assumption 5.1 hold, let the spectrum of $\hat{K}$ lie in $\mathcal{D}_{\hat{\kappa}}$, and let $p = \sum_j c_j T_{2j+1}$ be an odd polynomial of degree d with $\sup_{x\in\mathcal{D}_{\hat{\kappa}}} |p(x) - 1/x| \le \varepsilon < 1$. Consider the iteration $u_0 = 0$, $r_k = b - \hat{K}u_k$, $u_{k+1} = u_k + \tilde{v}_k$, where $\tilde{v}_k$ is the readout estimate of $p(\hat{K})r_k$ with relative error at most ζ. Then:*
(a) each round contracts the residual, $\|r_{k+1}\| \le \xi \|r_k\|$, with

$$\xi \;\le\; \varepsilon + (1+\varepsilon)\,\hat{\kappa}\,\zeta; \tag{12}$$

(b) if $\xi < 1$, then $\|r_R\| \le \varepsilon_f \|b\|$ after $R = \lceil \ln(1/\varepsilon_f)/\ln(1/\xi) \rceil$ rounds;
(c) each circuit makes d queries to $U_{\hat{K}}$ and succeeds with probability $p_{succ} \ge (1-\varepsilon)^2/\|c\|_1^2$; with amplitude amplification, the total number of block-encoding queries satisfies

$$Q \;\le\; c_{AA} \cdot R \cdot 2S \cdot d \cdot \frac{\|c\|_1}{1-\varepsilon}, \tag{13}$$

where c_{AA} is an absolute constant;
(d) the LCU normalisation satisfies $\|c\|_1 \ge \hat{\kappa} - \varepsilon$;
(e) the classical work consists of R residual evaluations (R products with $\tilde{K}$, i.e. $\mathcal{O}(R\,\mathrm{nnz}(\tilde{K}))$), R state preparations, and $\mathcal{O}(R(N+S))$ operations for the reconstruction of $\tilde{v}_k$.

Proof. (a) Since $r_{k+1} = r_k - \hat{K}\tilde{v}_k$, we have $r_{k+1} = (I - \hat{K}p(\hat{K}))\,r_k - \hat{K}\big(\tilde{v}_k - p(\hat{K})r_k\big)$. As $\hat{K}$ is symmetric with spectrum in $\mathcal{D}_{\hat{\kappa}}$, $\left\|I - \hat{K}p(\hat{K})\right\| = \max_{\lambda\in\sigma(\hat{K})} |\lambda|\,|1/\lambda - p(\lambda)| \le \varepsilon$. Moreover $\left\|p(\hat{K})\right\| \le \max_\lambda \big(1/|\lambda| + \varepsilon\big) \le \hat{\kappa} + \varepsilon \le (1+\varepsilon)\hat{\kappa}$, so that $\left\|\tilde{v}_k - p(\hat{K})r_k\right\| \le \zeta(1+\varepsilon)\hat{\kappa}\,\|r_k\|$, and $\left\|\hat{K}\right\| \le 1$. The triangle inequality gives (12). (b) follows by induction. (c) Because $\Pi\mathcal{W}^k\Pi = T_k(\hat{K})$, the LCU circuit is a block encoding of $p(\hat{K})/\|c\|_1$ and applies at most d walk steps. For every eigenvalue, $|p(\lambda)| \ge 1/|\lambda| - \varepsilon \ge 1 - \varepsilon$, so (8) gives $p_{\mathrm{succ}} \ge (1-\varepsilon)^2/\|c\|_1^2$. Amplitude amplification produces an accepted sample with at most $c_{\mathrm{AA}}/\sqrt{p_{\mathrm{succ}}} \le c_{\mathrm{AA}}\,\|c\|_1/(1-\varepsilon)$ circuit executions; each round uses $2S$ accepted samples. (d) $|p(1/\hat{\kappa})| \ge \hat{\kappa} - \varepsilon$ and $|p(x)| \le \sum_j |c_j||T_{2j+1}(x)| \le \|c\|_1$ for $|x| \le 1$. (e) is a count of the operations in the iteration. □

Remark 5.3 (constants and the empirical readout model)**.** For amplitude amplification with known success probability, $c_{\mathrm{AA}} \approx \pi/2$; our emulation charges $c_{\mathrm{AA}} = 1$. For the optimal polynomial, Table 10 gives $\|c\|_1/\kappa \in [1.6, 2.4]$ at $\varepsilon = 0.3$ and $[3.2, 3.7]$ at $\varepsilon = 10^{-6}$ for $\kappa \le 10^3$, consistent with the lower bound (d). With S samples per setting, the sampling error of the reconstructed vector scales as $\zeta = \mathcal{O}(\sqrt{N/S})$, so the worst-case bound (12) would require $S = \Theta(N\hat{\kappa}^2)$ for a constant contraction. The measurements in Section 8.8 do not show the factor $\hat{\kappa}$: on 45 (system, sample-count) settings with $\hat{\kappa}$ between 1.9 and 4.3×10^3, the readout contribution to the contraction is $c_0\sqrt{N/(2S)}$ with $c_0 = 1.10$ (10–90 % range 0.91–1.29) and a Spearman correlation of 0.09 between c_0 and $\hat{\kappa}$. A plausible explanation is that the worst case requires the residual to concentrate on the eigenvectors of the smallest eigenvalues and the readout error on those of the largest, which isotropic sampling noise does not do. Under this empirical model, $S = \Theta(N)$ suffices for a constant contraction, and with $\varepsilon = 0.3$ the contraction is $\xi \approx \max\{\varepsilon_{\mathrm{eff}}, c_0\sqrt{N/(2S)}\}$ with $\varepsilon_{\mathrm{eff}} \approx 0.25$–$0.30$, the larger of the two sources rather than their sum.

The following corollaries quantify the effect of the three methodological choices that enter Theorem 5.2: the separation of circuit precision from final precision, equilibration, and the choice of target system.

Corollary 5.4 (precision enters only through the number of rounds)**.** *For fixed circuit precision ε and fixed ζ with $\xi \le \xi_0 < 1$, the circuit depth d and the queries per accepted sample in Theorem 5.2 do not depend on the final accuracy ε_f, and Q grows like $\ln(1/\varepsilon_f)$. A single solve with direct readout to residual ε_f instead requires, in the worst case, a polynomial with $\varepsilon \le \varepsilon_f$, i.e. $d = \Theta(\hat{\kappa}\log(\hat{\kappa}/\varepsilon_f))$, and, under the empirical readout model, $2S \ge c_0^2 N/\varepsilon_f^2$ accepted samples. Combined with the precision law of Section 7.6, $\varepsilon_f \in [\tau/10, \tau]$ for IPM tolerance τ, refinement needs $R = \mathcal{O}(\log(1/\tau))$ rounds of constant depth, whereas direct readout needs $\Omega(N/\tau^2)$ samples per solve.*

Proof. d depends only on $(\hat{\kappa}, \varepsilon)$ by (9), and the queries per sample are $d\,\|c\|_1/(1-\varepsilon)$; ε_f enters only through R in Theorem 5.2(b). For direct readout the residual after one solve is at least the readout contribution $c_0\sqrt{N/(2S)}$, which must not exceed ε_f. □

For $N = 32$ and $\varepsilon_f = 10^{-8}$, the accuracy the IPM needs at CVXOPT's default tolerance, the direct-readout model requires 3.9×10^{17} accepted samples per solve; the refined solver used 7.3×10^3 per solve in the integrated runs of Section 8.5, on which the IPM converged. Refinement of quantum linear solves to escape the $1/\varepsilon$ dependence of tomography was introduced in [33, 42]; Corollary 5.4 makes the resulting depth and sample dependence explicit for the polynomial solver and connects it to the measured precision requirement of the IPM.

Corollary 5.5 (equilibration shortens the circuit)**.** *Let $\kappa = \kappa(\tilde{K}) \geq \hat{\kappa} = \kappa(D\tilde{K}D)$. For the optimal polynomial at fixed ε, the degrees satisfy*

$$\frac{d(\kappa,\varepsilon)}{d(\hat{\kappa},\varepsilon)} \;\simeq\; \frac{\kappa}{\hat{\kappa}} \cdot \frac{\log(\kappa/\varepsilon)}{\log(\hat{\kappa}/\varepsilon)} \;\geq\; \frac{\kappa}{\hat{\kappa}}$$

up to rounding, and the lower bound of Theorem 5.2(d) on the amplitude-amplification overhead of the unequilibrated system, $\|c\|_1 \geq \kappa - \varepsilon$, exceeds the measured overhead of the equilibrated one, $\|c\|_1 \leq 2.4\hat{\kappa}$ at $\varepsilon = 0.3$, by a factor of order $\kappa/\hat{\kappa}$. The bound (13) *on the queries per accepted sample therefore decreases by a factor of order $(\kappa/\hat{\kappa})^2$, at a classical cost of $\mathcal{O}(\mathrm{nnz}(\tilde{K}))$ per sweep and IPM iteration.*

Proof. From (9), $\log(1+a) - \log(1-a) = 2a + \mathcal{O}(a^3)$ with $a = 1/\kappa$, so $n = \frac{\kappa}{2}\log(\kappa/\varepsilon)\,(1 + \mathcal{O}(\kappa^{-1}))$; the ratio follows, and the remaining statements follow from Theorem 5.2(c,d) and Remark 5.3. □

Table 15 evaluates the corollary at the measured condition numbers: the median degree reduction at $\varepsilon = 0.3$ is 1.2×10^5 on `sparse_banded` and 1.2×10^4 on `dense_kappa`. Complexity bounds for quantum IPMs that are stated in terms of $\kappa(\tilde{K})$, or of the condition number of a reduced or normal-equation system, therefore overstate the cost of the linear solves on regular problems by this factor.

Corollary 5.6 (null-space versus augmented target)**.** *Let $\kappa_{\mathcal{N}}$ be the condition number of the scaled null-space matrix of* (4)*. The polynomial* (10) *reaches error ε with degree $d_{\mathcal{N}} \approx \frac{1}{2}\sqrt{\kappa_{\mathcal{N}}}\log(\kappa_{\mathcal{N}}/\varepsilon)$, $\|c\|_1 < \kappa_{\mathcal{N}}$ and $p_{succ} \geq (1-\varepsilon)^2/\kappa_{\mathcal{N}}^2$, so that*

$$\frac{d_{aug}}{d_{\mathcal{N}}} \;\approx\; \frac{2\hat{\kappa}\log(\hat{\kappa}/\varepsilon)}{\sqrt{\kappa_{\mathcal{N}}}\log(\kappa_{\mathcal{N}}/\varepsilon)}.$$

The null-space target therefore yields the shallower circuit when $\sqrt{\kappa_{\mathcal{N}}} \lesssim 2\hat{\kappa}$, up to logarithmic factors (and $\sqrt{\kappa_{\mathcal{N}}} \lesssim \frac{2}{3}\hat{\kappa}$ when the factor of about 3 for block-encoding the shifted matrix is included), and the cheaper one per accepted sample when $\kappa_{\mathcal{N}}^{3/2} \lesssim 2(\|c\|_1^{aug}/\hat{\kappa})\,\hat{\kappa}^2$, i.e. roughly $\kappa_{\mathcal{N}} \lesssim 2.5\,\hat{\kappa}^{4/3}$.

Proof. The truncation error bound of Section 4.4.4, $\ln(1/q) = 2/\sqrt{\kappa_{\mathcal{N}}} + \mathcal{O}(\kappa_{\mathcal{N}}^{-3/2})$, and $\sqrt{\kappa_{\mathcal{N}}}(\sqrt{\kappa_{\mathcal{N}}} + 1) \approx \kappa_{\mathcal{N}}$ give $d_{\mathcal{N}}$; the 1-norm and p_{succ} bounds follow as in Theorem 5.2(c), since $|1/x| \geq 1$ on $[a, 1]$. The ratios follow from (9) and the queries per sample $d\,\|c\|_1$. □

Because $\kappa_{\mathcal{N}}$ inherits the μ^{-2} growth of S while $\hat{\kappa}$ stays bounded on regular problems (Proposition 3.1), the criterion holds early in the IPM run and fails late; Section 8.7 confirms both regimes.

We now compare the cost of one Newton direction under the hybrid scheme with that of classical MINRES on the same equilibrated system. Let k denote the number of MINRES iterations per KKT

Table 1: Cost of one KKT solve to residual ε_f (Theorem 5.2, Corollaries 5.4–5.6, Proposition 5.7). $\hat{\kappa} = \kappa(D\tilde{K}D)$; k: MINRES iterations; $R = \mathcal{O}(\log(1/\varepsilon_f))$: refinement rounds; $\tilde{\mathcal{O}}$ hides factors logarithmic in $\hat{\kappa}$ and $1/\varepsilon$. Quantum entries count block-encoding queries; "empirical" refers to the readout model of Remark 5.3 ($S = \Theta(N)$), "worst case" to Theorem 5.2(a) ($S = \Theta(N\hat{\kappa}^2)$).

method	classical work per solve	quantum queries per solve
sparse LDL$^\mathsf{T}$ / Cholesky	one factorisation per IPM iteration (fill-dependent), $\mathcal{O}(\mathrm{nnz}(L))$ per solve	–
equilibrated MINRES	$k\,(2\,\mathrm{nnz}(\tilde{K}) + \mathcal{O}(N))$, with $k = \mathcal{O}(\hat{\kappa}\log(1/\varepsilon_f))$ in the worst case	–
polynomial QLSA, direct readout	$\mathcal{O}(N/\varepsilon_f^2)$ reconstruction	$\tilde{\mathcal{O}}(N\hat{\kappa}^2/\varepsilon_f^2)$, depth $\Theta(\hat{\kappa}\log(\hat{\kappa}/\varepsilon_f))$
polynomial QLSA, low precision + refinement (this work)	$R\,(2\,\mathrm{nnz}(\tilde{K}) + \mathcal{O}(N))$	$\tilde{\mathcal{O}}(RN\hat{\kappa}^2)$ empirical, $\tilde{\mathcal{O}}(RN\hat{\kappa}^4)$ worst case; depth independent of ε_f
optimal QLSA + refinement	$R\,(2\,\mathrm{nnz}(\tilde{K}) + \mathcal{O}(N))$	$\tilde{\mathcal{O}}(RN\hat{\kappa})$ empirical
null-space target + refinement	$R\,(2\,\mathrm{nnz}(\tilde{K}) + \mathcal{O}(n^2))$, including the completion (4)	$\tilde{\mathcal{O}}(R\,(n-p)\,\kappa_{\mathcal{N}}^{3/2})$ empirical

solve, R the number of refinement rounds, and $\bar{\nu} := 2\,\mathrm{nnz}(\tilde{K})/N + c_v$ the floating-point operations per row of one matrix–vector product and the associated vector updates.

Proposition 5.7 (cost per Newton direction)**.** *Let MINRES need k iterations per solve and the refined solver R rounds with $S = s_0 N$ accepted samples per setting, and let one block-encoding query, including its share of state preparation, cost t_q in units of classical floating-point operations. Then, up to lower-order terms,*

$$W_{MINRES} = k\,N\bar{\nu}, \qquad W_{hybrid} = R\,N\bar{\nu} + c_q\,R\,N\,\hat{\kappa}^{\gamma},$$

with $\gamma = 2$ and $c_q = \Theta\big(c_{AA}\,s_0\,t_q\log(\hat{\kappa}/\varepsilon)\big)$ for the post-selected LCU solver of Theorem 5.2, and $\gamma = 1$ for a QLSA with optimal query complexity $\mathcal{O}(\hat{\kappa}\log(1/\varepsilon))$ [17, 18] combined with the same readout and refinement. Consequently, the hybrid scheme is cheaper than MINRES only if

$$c_q\,\hat{\kappa}^{\gamma} \;<\; \Big(\frac{k}{R} - 1\Big)\,\bar{\nu}, \tag{14}$$

and the ratio W_{MINRES}/W_{hybrid} never exceeds k/R.

Proof. Each MINRES iteration costs one product with $\tilde{K}$ and $\mathcal{O}(N)$ vector operations. By Theorem 5.2(e) the hybrid scheme performs R such products plus $\mathcal{O}(R(N+S)) = \mathcal{O}(RN)$ reconstruction work, which we absorb into $\bar{\nu}$. By Theorem 5.2(c), its queries are at most $c_{\mathrm{AA}} R\,2s_0 N\,d\,\|c\|_1/(1-\varepsilon)$ with $d\,\|c\|_1 = \Theta(\hat{\kappa}^2\log(\hat{\kappa}/\varepsilon))$ by (9) and Theorem 5.2(d); for an optimal QLSA, the state is prepared with $\mathcal{O}(\hat{\kappa}\log(1/\varepsilon))$ queries and no separate post-selection overhead. Rearranging $W_{\mathrm{hybrid}} < W_{\mathrm{MINRES}}$ gives (14), and $W_{\mathrm{hybrid}} \geq RN\bar{\nu}$ gives the bound on the ratio. □

Interpretation. Proposition 5.7 has three consequences. First, any advantage of the hybrid scheme per Newton direction is polynomial and bounded by k/R, the ratio of classical matrix–vector products that it saves; this bound is consistent with the $\Omega(N/\varepsilon)$ tomography lower bound [58], which rules out an exponential advantage for the computation of a classical direction. Second, because $c_q\hat{\kappa}^\gamma \geq c_q\hat{\kappa}$ while k/R is at most of order $\hat{\kappa}$, condition (14) can only hold when the rows of $\tilde{K}$ are dense ($\bar{\nu}$ large) and the equilibrated condition number $\hat{\kappa}$ is moderate; on `sparse_banded`, with $\bar{\nu} \approx 30$ and at most $k/R \approx 160$ (MINRES needs 2 061 iterations per solve at $N = 79\,360$), it would require $c_q\hat{\kappa}^\gamma \lesssim 5 \times 10^3$, i.e. $c_q \lesssim 10$ for $\gamma = 1$ at $\hat{\kappa} \approx 5 \times 10^2$—a block-encoding query no more expensive than about ten classical floating-point operations, far below the costs implied by end-to-end resource estimates [19]. Third, the methodological components of Section 4 act directly on the left-hand side of (14): equilibration lowers $\hat{\kappa}$ by three to five orders of magnitude, low-precision refinement keeps R at about a dozen rounds independently of τ and N, and the optimal polynomial lowers the constant in c_q by 4.8× per accepted sample.

Instantiation. At the simulated sizes the condition fails for any c_q: for $N \leq 32$, MINRES needs 24–27 iterations per solve and the refined solver 10–19 rounds, so $k/R \leq 2.5$ (Fig. 15c). For `dense_kappa` at $N = 256$, MINRES needs $k = 198$ iterations per solve, the median $\hat{\kappa}$ is 4.2×10^3, and $\mathrm{nnz}(\tilde{K}) \approx 0.53N^2$, so $\bar{\nu} \approx N$. If $R \approx 13$ rounds suffice at this size, as Theorem 5.2 predicts under the empirical readout model with $S \propto N$, condition (14) becomes $N \gtrsim 3 \times 10^2\, c_q$ for $\gamma = 1$ and $N \gtrsim 1.2 \times 10^6\, c_q$ for $\gamma = 2$, holding k, R and $\hat{\kappa}$ at their $N = 256$ values. Since k grows with N on this family (from 24 at $N = 32$ to 198 at $N = 256$) while R does not, this estimate is conservative in k; it rests, however, on assumptions that we have not tested at scale—the empirical readout model beyond $N = 64$, a block encoding of a dense matrix whose query cost t_q does not grow with $\mathrm{nnz}(\tilde{K})$ (which requires a QRAM-type data structure whose classical update costs $\mathcal{O}(\mathrm{nnz}(\tilde{K}))$ per IPM iteration), and an optimal QLSA for $\gamma = 1$. For dense matrices the classical comparator could also be a dense factorisation ($N^3/3$ operations per IPM iteration); the same accounting gives the additional requirement $N^2 \gtrsim 6Rc_q\hat{\kappa}^\gamma$, which for $\gamma = 1$ amounts to $N \gtrsim 6 \times 10^2\sqrt{c_q}$. These estimates locate the regime in which the methodology could deliver a quantum advantage; they do not demonstrate one.

6 Experimental setup

6.1 Test instances

We use six synthetic QP families with known optimal solutions—five of them generated from a prescribed primal–dual solution with 30 % active constraints—together with two real-world collections (Table 2). The family `sanity` consists of three hand-constructed small QPs, `tiny_A`, `tiny_B` and `tiny_C` ($N = 4$–8), used for the contract check and as captured systems for the quantum experiments. The family `dense_kappa` has dense data with $n \in \{10, 20, 40, 80\}$, $\kappa(P) \in \{1, 10^3, 10^6\}$ and five seeds (60 instances), `dense_noeq` has no equality constraints (8 instances), and `sparse_scale` has random 5 % density (4 instances). The family `sparse_banded` has banded P and four nonzeros per row of G and A in *random* columns, with $n = 100$ to 25 600, i.e. $N = 310$ to 79 360, five seeds per size up to $n = 3\,200$ and two beyond (36 instances). The control family `sparse_lowfill` has the same sizes, but the four nonzeros lie in a window of 13 neighbouring columns (three seeds up to $n = 3\,200$ and one beyond; 21 instances). The random column pattern makes $\hat{G}^\mathsf{T}\hat{G}$ a random sparse graph whose Cholesky factor fills in almost completely, whereas the local pattern keeps it banded. The two sparse families were designed together: the first represents the regime in which an iterative

Table 2: Benchmark sets (session M). $N = n + p + m$ is the KKT dimension; for the synthetic families $m = 2n$ and $p = 0.1n$ or $0.2n$. The largest `sparse_banded` KKT matrix has 6.6×10^5 nonzeros.

family	instances	n	N	seeds	structure
`sanity`	3	2–4	4–8	–	`tiny_A`, `tiny_B`, `tiny_C`; dense
`dense_kappa`	60	10–80	32–256	5	dense, $\kappa(P) \in \{1, 10^3, 10^6\}$
`dense_noeq`	8	10, 40	30, 120	2	dense, $p = 0$
`sparse_scale`	4	25–200	77–620	1	random 5 % density
`sparse_banded`	36	100–25 600	310–79 360	5 / 2	4 nnz/row, random columns (high fill)
`sparse_lowfill`	21	100–25 600	310–79 360	3 / 1	4 nnz/row, local columns (low fill)
Maros–Mészáros	48	9–2 597	91–10 192	–	real-world convex QPs
SDPLIB	10	–	25–1 379	–	`truss`, `control`, `hinf`, `qap5`, `theta1`

solver can win, the second the control in which it should not. Instance identifiers encode the family, n, the exponent of $\kappa(P)$ and the seed, e.g. `dense_kappa_n10_k3_s0` or `sparse_banded_n800_k3_s1`.

As a real-world collection we use 48 instances of the Maros–Mészáros set [40] (the `.mat` files of [12]; KKT dimension 91–10 192 and up to 1.4×10^5 nonzeros), referred to by their original names (e.g. `QSHARE1B`, `CVXQP3_S`, `PRIMAL4`). They are converted to (1) as follows: rows with equal bounds become equality constraints (103 linearly dependent equality rows on six instances are removed by a pivoted QR decomposition), every finite bound becomes one inequality row, and free rows are dropped. P has fewer nonzeros than variables, and is hence singular, on 18 of the instances, and many are degenerate LP-like problems, so the regularity assumed in Proposition 3.1 does not hold. No optimal value is assumed: the objective of the unmodified solver serves as the reference. The instances were selected to cover the size range up to $N \approx 10^4$ at a bounded total run time; the eleven instances in that range on which `coneqp` itself ends without status *optimal* were excluded, so that the collection measures the linear solver rather than the IPM. Ten SDPLIB instances [10]—`truss1`–`truss4`, `hinf1`, `hinf2`, `control1`, `control2`, `qap5` and `theta1` (KKT dimension 25–1 379)—are solved with `conelp` as a secondary test (Appendix B).

6.2 Computational environment and sessions

All runs were executed in Kaggle sessions with four vCPUs and 31 GB of memory: the GPU sessions on an Intel Xeon at 2.0 GHz with a Tesla T4 (14.6 GB) through CuPy 14.0.1, and the CPU-only sessions on an Intel Xeon at 2.2 GHz. The implementation uses Python 3.12 with CVXOPT 1.3.2 (CHOLMOD for the reduced direct solver), SciPy 1.16.3 and NumPy 2.0.2; Qiskit 2.5.2 serves only to cross-check the statevector simulator. Table 3 lists the six sessions. Session M supplies every scaling, memory, GPU, real-world, tolerance-sweep and integrated-quantum result, as well as the HHL accuracy study; session A supplies the acceptance-test ablation, the spectrum study, the experiments on captured systems with the polynomial solver, optimisation-level refinement and SDPLIB, all of which are deterministic in their counts; sessions B–E are replications (Appendix C). Timings vary between sessions—on identical instances, the per-method medians of the CPU-only session E exceed those of M by 1.09–1.31×—so all ratios reported below are computed within one session.

6.3 Solvers and parameter settings

CVXOPT runs with absolute and feasibility tolerances 10^{-7}, relative tolerance 10^{-6} and at most 100 iterations. Table 4 lists the solvers. Unless stated otherwise, all Krylov methods use Ruiz

Table 3: Computing sessions. All sessions use the same instance generator and seeds. They differ in hardware, problem-size profile (*small*: one seed per size; *medium*: several), the fallback factorisation of Algorithm 1 and the KKT dimension from which the automatic dispatch uses the GPU. Sessions A–C ran the core experiment set without the Maros–Mészáros, tolerance-sweep, sampling-seed and null-space experiments.

id	hardware	profile	started	fallback	GPU from	role
M	T4 + Xeon 2.0 GHz	medium	26 Sep 2026	reduced, CHOLMOD	$N = 2 \times 10^4$	main session
A	Xeon 2.2 GHz	medium	21 Sep 2026	LU of $\tilde{K}$	–	acceptance tests, spectrum, captured systems, outer refinement, SDPLIB
B	T4 + Xeon 2.0 GHz	medium	21 Sep 2026	LU of $\tilde{K}$	$N = 8 \times 10^3$	replication (2 seeds)
C	T4 + Xeon 2.0 GHz	small	24 Sep 2026	LU of $\tilde{K}$	$N = 8 \times 10^3$	replication (full size range)
D	Xeon 2.2 GHz	small	26 Sep 2026	reduced, CHOLMOD	–	replication ($N \leq 9\,920$)
E	Xeon 2.2 GHz	medium	27 Sep 2026	reduced, CHOLMOD	–	full replication of M on the CPU

equilibration with six sweeps, the combined test (5) with $\eta = 0.1$ and $\tau_\omega = 10^{-10}$, a refinement budget of two rounds and classical fallback; ablation variants without scaling or without fallback are marked as such. The quantum solvers run without fallback and with a refinement budget of 60 rounds per solve unless stated otherwise.

6.4 Repetitions, budgets and accuracy

In session M, each (instance, method) pair is run twice; per instance we take the median of the two repetitions, and tables report the median over the instances of a size together with the range of the per-instance paired ratios. A predictive per-method budget skips a method at the next size when a power-law extrapolation of its last two timings exceeds 1.5×300 s; on the Maros–Mészáros collection, whose instances are heterogeneous, the extrapolation is disabled and a method is skipped only on instances larger than one on which it exceeded 420 s. Skipped cells are empty in the tables and count as unsolved in the performance profiles. All timings include assembly, equilibration, host–device transfers, verification, refinement and fallback, and a sampling thread records the growth of resident memory during every run (and of the CuPy memory pool on the GPU). On the synthetic families, all methods reach the same accuracy (relative gap $\leq 5.5 \times 10^{-7}$, objective error $\leq 3.2 \times 10^{-7}$, cone violation 0) with identical IPM iteration counts on every instance on which they ran, so the comparisons are between equally accurate solvers.

Table 4: Solvers. All replace the KKT solve through the `kktsolver` callback except the unmodified baseline.

name in this paper	description
LDL^T (CVXOPT, unmodified)	CVXOPT's own sparse LDL^T factorisation of the augmented system; the baseline
exact LU	dense or SuperLU factorisation of $\tilde{K}$ through the callback; verifies the solver contract
reduced system, CHOLMOD	CHOLMOD Cholesky factorisation of S in (3), dense Schur complement for A, one refinement step
lazy refactorisation	GMRES on the reduced system, preconditioned by the factorisation of a previous IPM iteration
equilibrated MINRES (SciPy)	SciPy MINRES on $D\tilde{K}D$ with the stopping tests of Section 4.1; used for the acceptance-test ablation
equilibrated MINRES, baseline kernel	own array-module MINRES (NumPy or CuPy), direct transcription
equilibrated MINRES, latency-optimised kernel	kernel of Section 4.3 on CPU or GPU; *auto*: CPU below and GPU above the dispatch threshold; *switch*: direct solver after the first rejected direction
null-space CG	CG on (4) with a dense basis Z ($n \leq 3\,200$; iteration counts)
HHL	gate-level phase-estimation HHL (Section 4.4.2), n_c clock qubits
LCU polynomial solver	walk-level LCU with the CKS or the optimal polynomial on the augmented target, or the shifted Chebyshev polynomial on the null-space target (Section 4.4.3)

7 Results: classical solvers

This section first fixes the acceptance test on the dense families, then examines one IPM run in detail to explain the Krylov iteration counts, and reports the scaling behaviour on the two sparse families, the GPU kernel, the real-world QPs, the precision requirement of a Newton direction, and the alternatives that were evaluated and not adopted.

7.1 Acceptance test and the role of equilibration

Table 5 and Fig. 2 summarise the ablation on the 68 instances of `dense_kappa` and `dense_noeq` (136 runs per method). Three observations follow.

Equilibration, not the choice of test, determines reliability. Without equilibration no test is reliable: the relative-residual test needs classical fallback in 38 % of runs, the backward-error test in 46 % and the block test in 50 %, at 2 100–5 000 Krylov iterations per run. With Ruiz equilibration, every test with a tolerance of 10^{-10} or tighter, and the block test with $\eta \leq 0.1$, is fallback-free on 100 % of runs at 650–1 000 iterations.

A backward-error tolerance must lie far below the IPM tolerance. A backward-error tolerance of 10^{-6} reaches the final IPM tolerance in only 26 % of runs and 10^{-8} in 78 %, because by Lemma 4.1 these tests certify only a forward error of order $10^{-5}\kappa$ and $10^{-7}\kappa$, which exceeds what the IPM tolerates (Section 7.6) as soon as $\kappa(\tilde{K}) \gtrsim 10$.

The certificate costs 22–34 %. The combined test (5) is fallback-free at 868 Krylov iterations per run, against 648 for the block test alone and 712 for the relative-residual test, which is attainable on these instances; the additional work is the price of the backward-error certificate. The combined test is used in all subsequent experiments.

Table 5: Acceptance-test ablation on 68 dense QP instances × 2 repetitions (session A). Session M reproduces every fallback-free rate, and the Krylov medians of the fallback-free tests to within 2 %. *acc.*: final tolerance reached (fallback allowed); *ff*: fallback-free; *fb*: total fallbacks; Krylov: median total iterations per run; wall: median seconds per run.

test	scaling	acc. %	ff %	fb	Krylov	wall [s]
relative residual 10^{-6}	none	100	62	432	2 444	0.152
backward error 10^{-10}	none	87	54	376	5 012	0.332
block $\eta = 0.1$	none	99	50	444	2 100	0.116
relative residual 10^{-6}	Ruiz	100	100	0	712	0.046
backward error 10^{-6}	Ruiz	26	21	284	1 836	0.263
backward error 10^{-8}	Ruiz	78	75	186	952	0.063
backward error 10^{-10}	Ruiz	100	100	0	856	0.058
backward error 10^{-12}	Ruiz	100	100	0	1 016	0.063
block $\eta = 0.5$	Ruiz	96	91	38	1 064	0.077
block $\eta = 0.1$	Ruiz	100	100	0	648	0.044
block $\eta = 0.01$	Ruiz	100	100	0	752	0.047
combined ($\eta = 0.1$, $\tau_\omega = 10^{-10}$)	Ruiz	100	100	0	868	0.054
combined, latency-optimised kernel	Ruiz	100	100	0	880	0.035
adaptive backward error (NT spread)	Ruiz	96	96	0	832	0.064
LDL^T (CVXOPT, unmodified)	–	100	100	0	0	0.0012
reduced system, CHOLMOD	–	100	100	0	0	0.0064

7.2 Conditioning and Krylov work along the IPM run

Fig. 3 follows one IPM run on `sparse_banded_n800_k3_s1` ($N = 2\,480$, session A) with every $\tilde{K}(W)$ and right-hand side captured.

The equilibrated system remains well conditioned. The NT spread $\max w / \min w$ grows from 1 to 1.4×10^6, whereas $\kappa(D\tilde{K}D)$ stays between 80 and 178 (panel a), as Proposition 3.1 predicts. The same capture on `sparse_banded_n800_k3_s4` in session M gives $\kappa(D\tilde{K}D)$ between 111 and 703 at an NT spread of up to 3.5×10^5; on all `sparse_banded` instances on which it was measured, the equilibrated condition number stays below about 7×10^2.

The iteration count is governed by the eigenvalue distribution. MINRES iterations nevertheless grow from 92 to 1 880 per IPM iteration (panel b), because the share of eigenvalues within 10 % of ± 1 falls from 76 % to 54 % and up to 38 eigenvalues of magnitude below 0.1 appear. Two further measurements are relevant. After the first IPM iteration the combined test costs 1.17–1.56× the iterations of the backward-error test alone, and in every solve the binding block is the equality block y, whose right-hand side is of order 10^{-15} once the iterates are feasible (Section 4.1). The matrix-free reduced system needs 0.57× the iterations early in the run but 2.8× late in the run, which confirms the augmented system as the appropriate target.

Deflation would help; harvested recycling does not. Panel (c) bounds what deflation could achieve: exact removal of the k smallest-magnitude eigenpairs reduces the iterations 1.7× ($k = 10$), 3.7× ($k = 50$) and 6.2× ($k = 100$). A recycled-MINRES prototype, with harmonic Ritz vectors harvested from the Lanczos process ($k = 40$–80) and carried across the solves of one run, achieved only 1.38–1.63× fewer iterations at 6× the CPU time, because the harvested vectors approximate the interior eigenvalues poorly; it was not adopted.

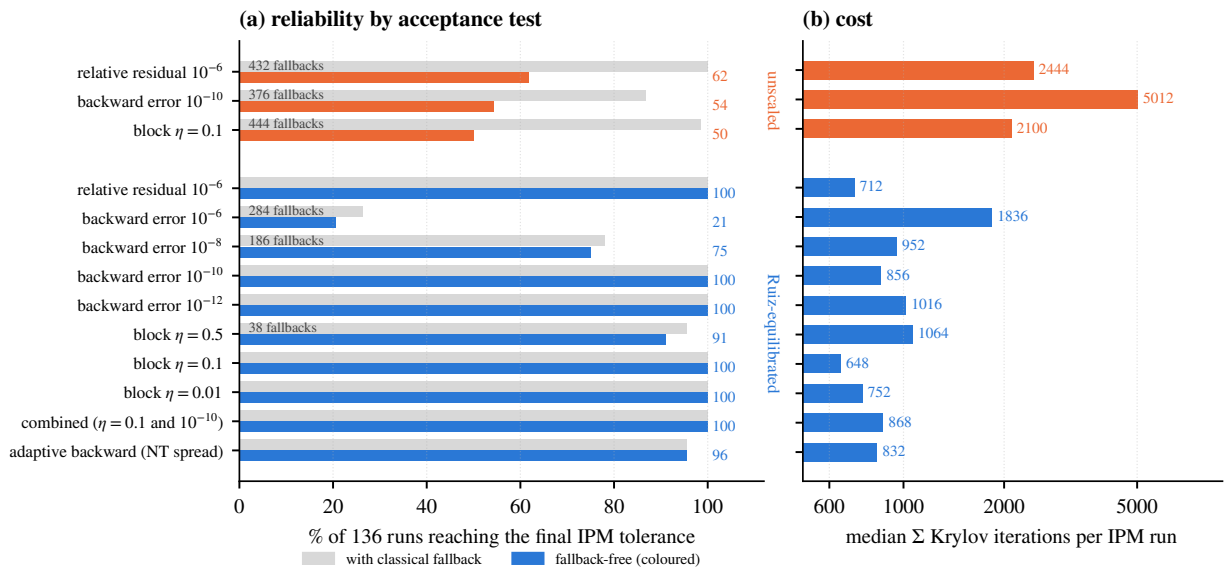


Fig. 2: Acceptance-test ablation (Table 5). (a) Share of the 136 runs reaching the final IPM tolerance with classical fallback (grey) and without it (coloured; numbers at the bar ends); (b) median Krylov work per run. Red: unscaled; blue: Ruiz-equilibrated.

7.3 Scaling behaviour

Table 6 and Fig. 4 report the whole-run wall time on the two sparse families (session M; five instances per size on `sparse_banded` up to $N = 9\,920$).

Cost. On `sparse_banded`, the direct solvers scale like $N^{2.3}$ (LDL^T) and $N^{1.9}$ (CHOLMOD) over the measured range and like $N^{3.2}$–$N^{3.5}$ between the two largest sizes they completed; the predictive budget stopped them beyond $N = 19\,840$. Equilibrated MINRES on the CPU scales like $N^{1.2}$, because its total Krylov iteration count grows like $N^{0.55}$ (from 1 500 to 39 000 per run).

Crossover. MINRES is slower than the sparse LDL^T factorisation below $N \approx 5\,000$ (0.06× at $N = 310$), breaks even at $N = 4\,960$ (paired ratio 1.26; 0.97–1.47 over five instances), and is 3.5× (3.1–4.5×) faster at $N = 9\,920$ and 9.4× (8.5–10.3×, two instances) at $N = 19\,840$, where it also outperforms the CHOLMOD-based reduced solver by 13×. The instance-to-instance spread is below ±30 % at every size beyond $N = 1\,240$.

Low-fill control. On `sparse_lowfill` the direct solvers win at every size up to $N = 39\,680$ (the reduced CHOLMOD solve is 1.9–5.2× faster than CPU MINRES from $N = 4\,960$ on); only at $N = 79\,360$ does GPU MINRES overtake both direct solvers (3.7×), where the dense $p \times p$ Schur complement of the reduced solver and the LDL^T factorisation both become expensive. The speed-up on `sparse_banded` is therefore a statement about fill-in, not about iterative methods in general.

Memory. Fill-in costs memory as well as time (Fig. 4c): at $N = 19\,840$ on `sparse_banded`, resident memory grows during a run by 390 MB (LDL^T), 472 MB (lazy refactorisation) and 722 MB (CHOLMOD), against 47 MB for CPU MINRES; at $N = 79\,360$, CPU MINRES needs 218 MB and GPU MINRES 134 MB of device memory. On `sparse_lowfill` the direct solvers are frugal up to $N = 19\,840$ but grow by 287 MB (LDL^T) and 2.6 GB (CHOLMOD with the dense Schur complement) at $N = 79\,360$. Below $N \approx 5\,000$ the growth is below the sampling resolution because Python's allocator reuses memory between runs.

Performance profiles. Fig. 5(a,b) aggregates all 57 sparse instances. On `sparse_banded`, LDL^T is the fastest method on 56 % of the instances (the small ones) but solves only 89 % within the

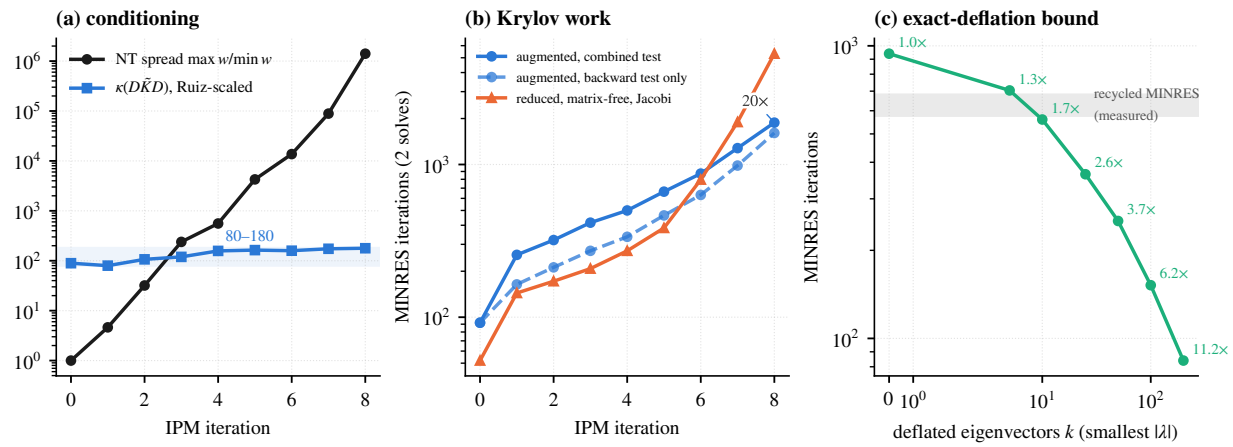


Fig. 3: One captured IPM run on `sparse_banded_n800_k3_s1` ($N = 2\,480$). (a) The NT scaling spreads over six orders of magnitude while κ of the Ruiz-equilibrated KKT matrix stays below 180. (b) MINRES iterations for the two solves of each IPM iteration under the combined test, under the backward-error test alone, and on the matrix-free reduced system. (c) Iterations at the last IPM iteration after exact deflation of the k smallest-$|\lambda|$ eigenpairs (an upper bound for any recycling scheme); the grey band marks what the recycled-MINRES prototype achieved.

budget; CPU MINRES and the automatic CPU/GPU dispatch solve all of them, and the dispatch is within 2× of the fastest method on 50 %. On `sparse_lowfill`, LDL$^\mathsf{T}$ is within 2× of the fastest on 95 % of the instances and CPU MINRES on 5 %.

Replication. Sessions B–E reproduce every crossover (Appendix C): the MINRES/LDL$^\mathsf{T}$ ratio at $N = 9\,920$ is 3.9, 3.1, 4.0 and 4.1× in sessions B, C, D and E, at 19 840 it is 9.1, 9.5 and 9.1× in sessions B, C and E, and the GPU/CPU ratio at 79 360 is 3.7× in both GPU replications.

7.4 GPU kernel

Fig. 6 isolates the kernel.

The baseline kernel is latency-bound. The GPU sparse mat-vec takes a constant ≈ 0.07 ms at every size and is 13.4× faster than the CPU at $N = 79\,360$ (panel a, dotted), but the baseline kernel spends 0.68–0.73 ms per MINRES iteration independently of N—about 90 % of its time in synchronisation and launch overhead, in agreement with (7).

The latency-optimised kernel. The kernel of Section 4.3 costs 0.53–0.62 ms per iteration and is 1.14–1.29× faster than the baseline kernel at every size on the GPU, and up to 1.21× faster on the CPU, where the gain comes from the CSR-level equilibration sweep and the sparser checks (it is 15 % slower at $N = 310$). Its Krylov iteration count is within 3.5 % of that of the baseline kernel (the adaptive checks overshoot slightly), and it is fallback-free on every run. The whole-run GPU speed-up over the CPU is 1.02× at $N = 19\,840$, 1.90× at 39 680 and 3.75× (3.62–3.87×) at 79 360 (panel b); below $N \approx 10^4$ the GPU is slower. The automatic dispatch at $N = 2 \times 10^4$ therefore uses the CPU up to 19 840 and the GPU beyond, and is the fastest MINRES variant at every size (Fig. 5a).

Mixed precision does not help here. Single-precision MINRES inside double-precision refinement, evaluated offline, needs 1.36× more iterations, because each restart discards the Krylov space, which a latency-bound kernel cannot recover.

Table 6: Median wall time [s] of the whole IPM run (session M; per instance the median of two repetitions, then the median over the instances of a size). Empty: skipped by the predictive budget (> 450 s predicted) or above the exact-LU limit. Last column: median and range over instances of the paired ratio LDL$^\mathsf{T}$/MINRES on the CPU (latency-optimised kernel).

	direct			MINRES, latency-optimised		MINRES, baseline kernel		LDL$^\mathsf{T}$/MINRES
N	LDL$^\mathsf{T}$	reduced	lazy refact.	CPU	GPU	CPU	GPU	CPU
`sparse_banded` (high fill; 5 instances per size up to $N = 9\,920$, 2 beyond)								
310	0.0061	0.043	0.10	0.090	1.00	0.085	1.14	0.06× (0.06–0.14)
620	0.016	0.067	0.18	0.18	1.69	0.21	1.91	0.09× (0.07–0.11)
1 240	0.058	0.13	0.24	0.28	2.19	0.29	2.54	0.19× (0.18–0.23)
2 480	0.23	0.31	0.47	0.51	3.33	0.52	3.79	0.49× (0.44–0.52)
4 960	1.35	1.49	1.60	1.07	4.74	1.14	5.33	1.26× (0.97–1.47)
9 920	9.80	10.9	11.5	2.73	6.94	3.06	8.39	3.50× (3.12–4.53)
19 840	88.1	123	118	9.47	9.26	10.8	11.2	9.42× (8.54–10.31)
39 680	–	–	–	24.2	12.7	27.9	15.3	–
79 360	–	–	–	85.0	22.5	100	28.3	–
`sparse_lowfill` (low fill; 3 instances per size up to $N = 9\,920$, 1 beyond)								
2 480	0.077	0.15	0.27	0.65	4.77	0.78	5.86	0.12× (0.10–0.15)
4 960	0.39	0.30	0.42	1.54	6.65	1.72	8.32	0.25× (0.16–0.30)
9 920	2.54	1.53	2.00	3.92	9.81	4.49	12.4	0.59× (0.35–0.85)
19 840	6.66	5.00	5.67	13.8	13.5	15.3	16.8	0.48×
39 680	32.9	18.0	18.4	35.1	18.0	38.9	22.6	0.94×
79 360	67.6	64.9	72.9	60.9	17.5	70.6	21.8	1.11×

7.5 Real-world QPs

Table 7, Fig. 7 and Fig. 5(c) report the 48 Maros–Mészáros instances (session M).

Every method is accurate; the direct solver is fastest. All methods reach the independent tolerances on every instance, with objective values within 6.3×10^{-9} of those of the unmodified solver and, on 46 of 48 instances, the same IPM iteration count. However, CVXOPT's own LDL$^\mathsf{T}$ factorisation is the fastest method on 40 instances, the reduced CHOLMOD solve on 6, and a MINRES variant on 2 (`GOULDQP3` and `PRIMAL4`, where MINRES needs no fallback). Equilibrated MINRES is a median 31× slower than LDL$^\mathsf{T}$, and its performance profile reaches $\rho = 0.25$ only at $\tau = 16$.

The equilibrated matrices are ill-conditioned. MINRES needs classical fallback on 26 instances (403 of its 1 520 solves). An offline diagnostic on eight instances with $N \le 1\,565$ (Fig. 7b, Table 15) recorded $\kappa(D\tilde{K}D)$ along the run: it reaches 2.4×10^6 (`QE226`) to 4.2×10^{15} (`QSC205`) on the five instances that need fallback, where MINRES reaches its iteration cap, and stays between 7.3×10^2 (`DUAL1`) and 1.0×10^7 (`CVXQP1_S`) on the three that do not, against at most about 7×10^2 on `sparse_banded`. These problems violate the premises of Proposition 3.1—P is singular on 18 of the 48 and the active sets are degenerate—so the boundedness that makes equilibration sufficient on the synthetic families does not hold, and an unpreconditioned Krylov method stalls.

Switching to the direct solver limits the damage but does not close the gap. Using MINRES until its first rejected direction and the direct solver thereafter reduces the fallbacks from 403 to 43 and the total MINRES time from 721 s to 214 s (median per instance from 3.2 to 0.86 s), while MINRES still delivers 63 % of all KKT solves (median 80 % per instance). It remains a median 21× slower than LDL$^\mathsf{T}$. Its 16 rejected directions are directions of the full-matrix LU decomposition used after the switch that miss the backward-error test at κ up to 10^{15}.

The GPU does not help at these sizes. On the eleven instances with $2\,400 \le N \le 3\,962$ on which it ran, GPU MINRES is 1.9–8.8× (median 7.4×) slower than CPU MINRES with the same Krylov counts, as the latency model predicts below $N \approx 2 \times 10^4$; the budget stopped it beyond $N = 3\,962$.

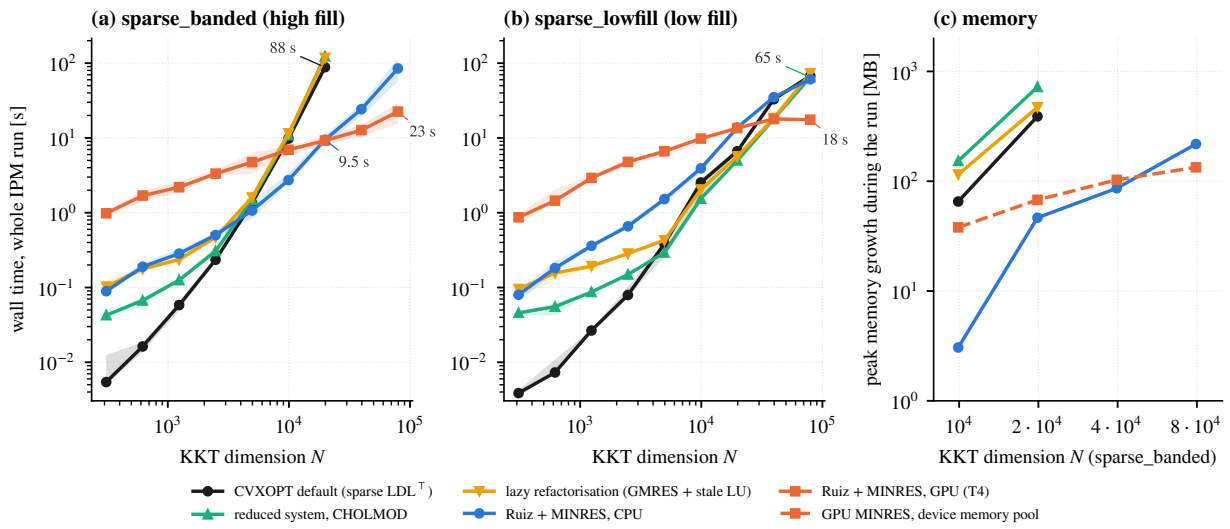


Fig. 4: (a, b) Whole-run wall time versus KKT dimension (Table 6); bands: range over the instances of a size. The iterative methods win under high fill-in (a) and lose under low fill-in (b) except at the largest size on the GPU. (c) Growth of resident memory during the run on `sparse_banded`, and the CuPy device-memory pool of GPU MINRES.

Table 7: Maros–Mészáros QPs (48 instances, $N = 91$–$10\,192$, session M). *strict*: tolerances reached without fallback or rejected direction; *inst. fb*: instances with at least one fallback; *same its*: share of instances with the IPM iteration count of the unmodified solver; speed-up: median over instances of the paired ratio to the unmodified LDL$^\mathsf{T}$; fastest: number of instances on which the method was the fastest of the five CPU methods.

method	inst.	strict %	inst. fb	fallbacks	same its %	median [s]	total [s]	speed-up	fastest
LDL$^\mathsf{T}$ (CVXOPT, unmodified)	48	100	0	0	100	0.072	25.9	1	40
reduced system, CHOLMOD	48	100	0	0	100	0.18	42.5	0.34	6
lazy refactorisation	48	94	3	18	98	0.42	50.0	0.15	0
Ruiz + MINRES, CPU	48	46	26	403	96	3.17	721	0.032	0
Ruiz + MINRES, switch to direct	48	46	26	43	98	0.86	214	0.048	2
Ruiz + MINRES, GPU ($N \geq 2\,400$)	11	55	5	94	100	76.6	1 432	0.006	–

7.6 The precision requirement of a Newton direction

Two experiments quantify the requirement that any inexact or quantum solver must meet (Fig. 8).

Injected direction error. The exact direction is perturbed by a random relative error of prescribed size before it is returned to the IPM (no acceptance test, no fallback). With an isotropic relative error of 10^{-7} or 10^{-8}, all 15 instances reach the final tolerance in the usual 6–7 iterations; with 10^{-6} none does within 100 iterations, and block-wise relative errors of 10^{-3} or larger fail as well. The threshold coincides with CVXOPT's own tolerance of 10^{-7}: the IPM cannot converge below the accuracy of its directions.

The requirement scales with the IPM tolerance. Table 8 and Fig. 8(b) repeat the experiment on 30 dense instances with CVXOPT's tolerances set to $\tau \in \{10^{-8}, 10^{-6}, 10^{-4}\}$ (absolute and feasibility tolerance τ, relative tolerance 10τ) and the acceptance thresholds scaled accordingly (primal and dual residuals $\leq 10\tau$, gap and objective error $\leq 100\tau$; $\tau = 10^{-7}$ recovers the defaults). The largest isotropic error tolerated on every instance is 10^{-7} at $\tau = 10^{-6}$ and 10^{-5} at $\tau = 10^{-4}$, and 10^{-8} is tolerated on 29 of 30 instances at $\tau = 10^{-8}$—between $\tau/10$ and τ at every level. Block-wise errors of

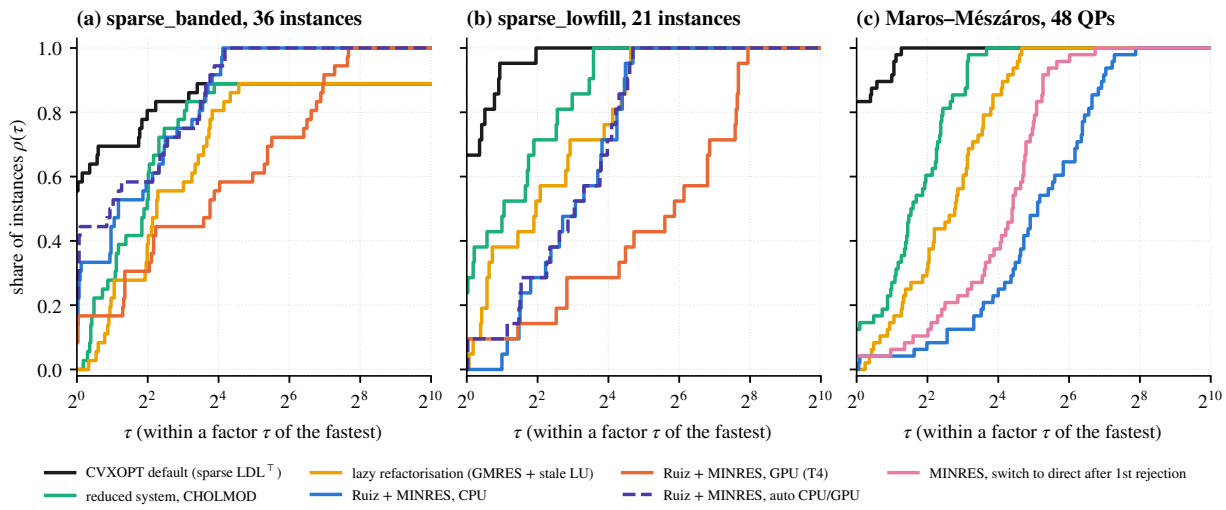


Fig. 5: Performance profiles [21] of whole-run wall time (session M): $\rho(\tau)$ is the share of instances on which a method reaches the final tolerance within τ times the time of the fastest method on that instance; runs skipped by the budget count as unsolved. (a, b) Synthetic sparse families; (c) Maros–Mészáros QPs (Section 7.5).

Table 8: Share of 30 dense instances [%] reaching the final tolerance when every Newton direction carries an injected relative error δ, at IPM tolerance τ (session M; the $\tau = 10^{-7}$ column is the injected-error experiment at CVXOPT's defaults on 15 instances). Solver rows: share and, in parentheses, median IPM iterations.

error model	δ	$\tau = 10^{-8}$	$\tau = 10^{-7}$	$\tau = 10^{-6}$	$\tau = 10^{-4}$
isotropic	10^{-8}	97	100	100	100
isotropic	10^{-7}	0	100	100	100
isotropic	10^{-6}	0	0	93	100
isotropic	10^{-5}	0	–	0	100
block-wise	10^{-3}	0	0	0	27
block-wise	10^{-2}	0	0	0	0
Ruiz + MINRES, combined test		100 (6)	–	100 (5)	100 (3)
unmodified solver		100 (6)	–	100 (5)	100 (3)

10^{-3} are tolerated only at $\tau = 10^{-4}$, and then on 27 % of the instances. The combined-test MINRES reaches every tolerance with the iteration count of the unmodified solver. The precision a KKT solver must deliver is therefore set by the optimisation tolerance, which is the quantity to budget for when a quantum or other inexact solver is costed (Corollary 5.4).

Optimisation-level refinement. Following [27], the QP is solved to a loose tolerance of 10^{-3}, the problem is shifted and scaled around the current point, and the correction QP is solved again, five times. With exact inner solves, or with MINRES accepted at a backward error of 10^{-4}, the error on the original problem falls from 1.7×10^{-4} to 5×10^{-9}, 5×10^{-13} and 10^{-15} (9/9 instances below 10^{-7}); with an injected forward error of 10^{-4} or 10^{-3} it stalls (2/9 and 1/9 instances), and with HHL directions without fallback it stalls as well (1/3 and 0/3). The correction QPs are badly conditioned—in session M, which also records the equilibrated condition number, their KKT matrices reach $\kappa(D\tilde{K}D) = 7 \times 10^6$ in the median and 10^{10} at most in the MINRES runs—yet residual-controlled MINRES still converges. Outer refinement thus relaxes the *residual* accuracy that

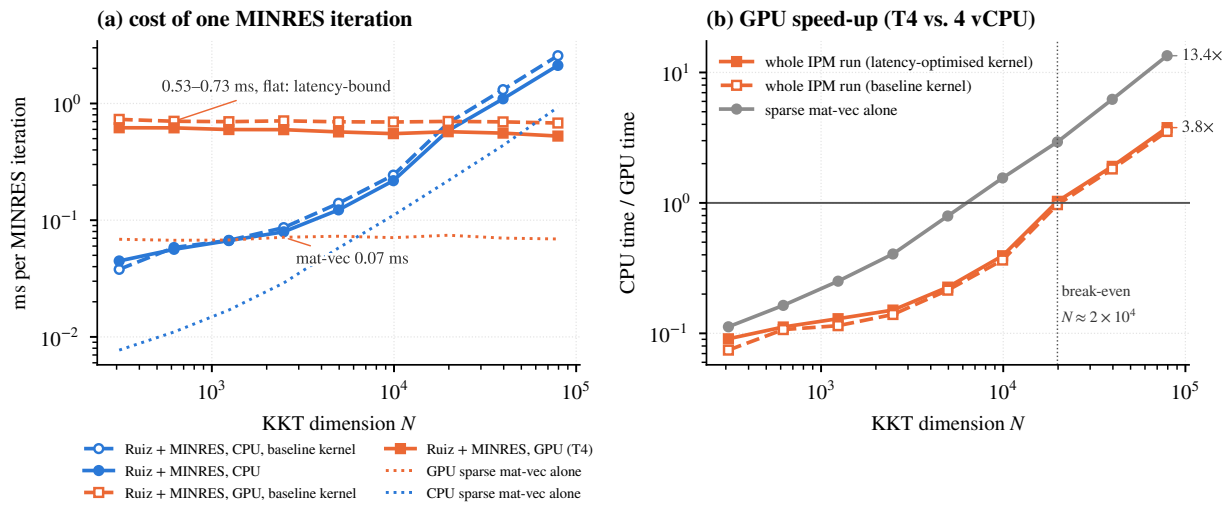


Fig. 6: GPU kernel on a Tesla T4 (session M). (a) Milliseconds per MINRES iteration (solve and acceptance checks) for the baseline (dashed) and latency-optimised (solid) kernels on CPU and GPU, with the sparse mat-vec alone (dotted). (b) Speed-up of the GPU over the 4-vCPU host for the whole IPM run and for the mat-vec alone.

a classical iterative solver must reach inside the IPM, but it does not make a forward-error-limited solver sufficient.

7.7 Alternatives evaluated and not adopted

Table 9 lists the alternatives that were implemented or prototyped and the measurement that led us not to adopt them. The most instructive negative results concern the reduced target system (Section 7.2), recycled MINRES and randomised Nyström preconditioning [16]: on our families the effective dimension of the active-constraint term $\hat{G}^{\mathsf{T}}\hat{G}$ equals the number of active constraints (480 of $n = 800$), and removing even the 100 dominant eigenpairs of S leaves $\kappa(S) = 1.5 \times 10^5$, so the low-rank premise fails. The method should suit problems with few active constraints (e.g. portfolio selection or support-vector machines), which we have not tested.

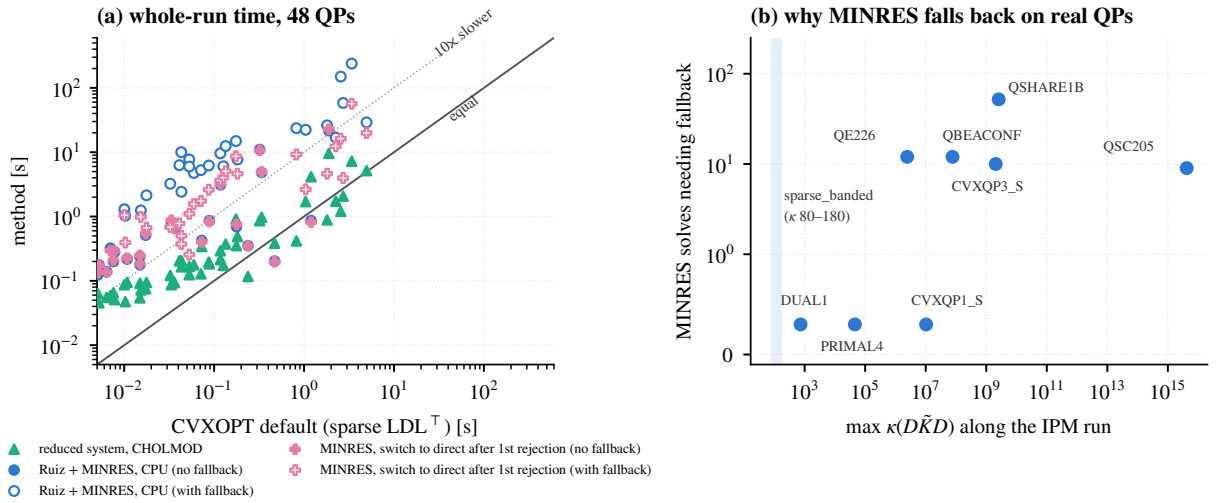


Fig. 7: Maros–Mészáros QPs. (a) Whole-run time of three methods against CVXOPT's LDL^{T} on the 48 instances; hollow markers: at least one classical fallback. (b) Offline diagnostic on eight instances: the largest condition number of the Ruiz-equilibrated KKT matrix along the IPM run against the number of MINRES solves that needed fallback; the band marks `sparse_banded`.

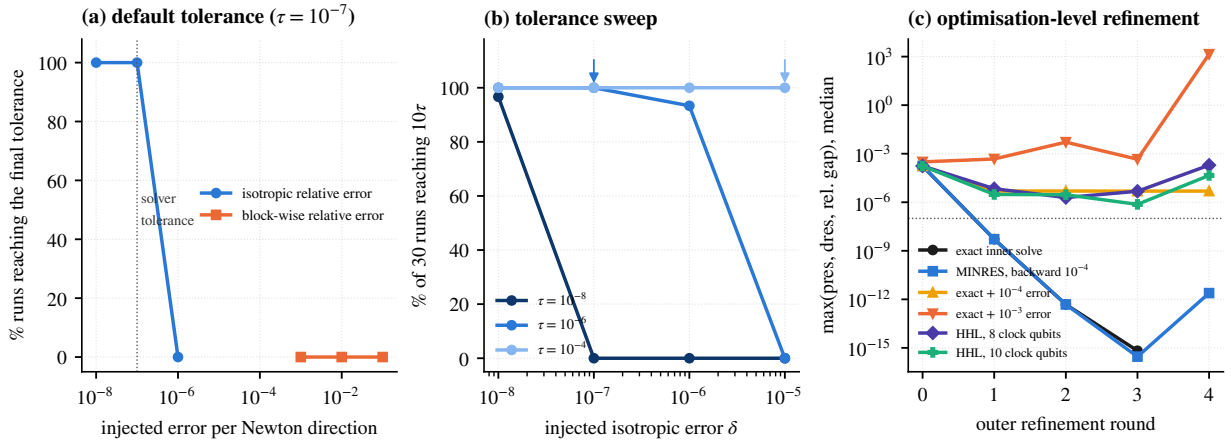


Fig. 8: (a) Share of IPM runs reaching the final tolerance when every Newton direction carries an injected relative error, at CVXOPT's default tolerance (15 instances). (b) The same for isotropic error at three IPM tolerances τ (30 instances); arrows mark the largest error tolerated on every instance. (c) Optimisation-level iterative refinement with different inner KKT solvers (session A; medians over 9 instances, HHL: 3; dotted: 10^{-7}).

Table 9: Alternatives evaluated and not adopted (measured on the benchmark of Section 6 or on its captured KKT sequences).

method	measurement
warm start of the combined solve from the affine direction	$\leq 4\,\%$ fewer Krylov iterations on sparse, more on dense instances; a naive variant accepts the affine direction itself under $\eta = 0.1$ and diverged on 2/35 instances
switch to the exact solver after the first rejected direction	never triggered on the synthetic families; on the Maros–Mészáros collection it reduces fallbacks from 403 to 43 and time 3.4× but remains 21× slower than LDL^T (Section 7.5)
CG on the null-space system (4) (dense basis)	2.1× fewer Krylov iterations than augmented MINRES on the dense families, 0.65–1.24× on the sparse families; the dense basis makes it 23–29× slower at $N = 9\,920$
lazy refactorisation (GMRES + stale factorisation)	half of the factorisations saved on small instances, none at scale (the scaling changes too fast); same cost as the direct solver at $N = 19\,840$
reduced system, matrix-free, Jacobi-scaled	0.57× the iterations early, 2.8× late in the run
square-root system $B = [L^\mathsf{T}; \hat{G}]$ with LSQR	$\kappa(B) = \sqrt{\kappa(S)}$ helps only when $\kappa(P)$ dominates; loses late in the run
recycled MINRES (harmonic Ritz, $k = 40$–80)	1.38–1.63× fewer iterations, 6× more CPU time (exact-deflation bound: 3.7–6.2×)
randomised Nyström preconditioning	effective dimension of $\hat{G}^\mathsf{T}\hat{G}$ equals the number of active constraints ($0.6n$); rank-100 removal leaves $\kappa(S) = 1.5 \times 10^5$
single-precision MINRES + double-precision refinement	1.36× more iterations (Krylov space discarded at each restart)
relaxed block floor ϕ_b ($10^{-9} \to 10^{-8}$)	$6\,\%$ fewer iterations on 51 instances (a tolerance relaxation, not adopted as default)
dequantised (sampling-based) solver [56]	$99\,\%$ of the singular directions needed for $30\,\%$ accuracy; stable rank $0.13N$: no low-rank structure to exploit

8 Results: quantum solvers

8.1 Setup and scope of the simulation

All results in this section are classical statevector simulations of ideal circuits with oracle block encoding and state preparation; they establish algorithmic feasibility and relative cost, not a speed-up over classical solvers. The captured KKT systems are equilibrated matrices from the `sanity` and dense families and from SDPLIB ($N \leq 64$, padded to 2^{n_d}), and the integrated runs use eight instances inside `coneqp`: `tiny_A`, `tiny_B`, `tiny_C`, `dense_noeq_n10_k0_s0`, `dense_kappa_n10_k0_s0`, `dense_kappa_n10_k0_s1`, `dense_kappa_n10_k3_s0` and `dense_kappa_n10_k3_s1` ($N \leq 32$). The walk-level simulation of the polynomial solver is cross-checked in three ways: the identity $\Pi\mathcal{W}^k\Pi = T_k(\hat{K})$ is asserted numerically; an explicit index-register statevector agrees with the accumulated form to 10^{-16}; and the cached spectral evaluation used for the second right-hand side and the refinement rounds agrees with the walk simulation to within 10^{-11} on every system (logged per solve). A simulation cap of 4×10^5 walk steps makes circuits beyond the budget fail with an explicit label; no classical result is ever substituted. The polynomial experiments on captured systems (Sections 8.3 and 8.4) come from session A; the HHL accuracy study, the integrated runs, the sampling-seed sweep and the null-space study from session M, in which the statevector simulator runs on the GPU. Because instances, matrices and quantum samples are seeded deterministically, every integrated configuration has the same strict-success count in sessions A, M and E.

8.2 HHL: accuracy and behaviour inside the IPM

Fig. 9(a) shows the forward error of the HHL output with ideal readout on 20 captured systems—five IPM iterations each of `tiny_A`, `tiny_B`, `tiny_C` and the SDPLIB instance `truss1`, with $\kappa(\tilde{K})$ from 2.0 to 7.4×10^{10}—as a function of the clock register. Without scaling, the median error is ≥ 0.9 up to 6 clock qubits and 1.5–2.7×10^{-2} at 10; Ruiz equilibration reduces it to 4.6×10^{-3} (κ-known mode) and 1.2×10^{-3} (resolution mode) at 10 clock qubits—four orders of magnitude short of the 10^{-7} required by the IPM (Section 7.6). Finite-shot readout adds error: at 10 clock qubits with equilibration, the median error is 1.8×10^{-2} (κ-known) and 0.15 (resolution) with 10^5 accepted samples per setting, and 3.3×10^{-2} and 0.55 with 10^6 raw shots. Inside the IPM (Table 12), with a budget of 4 refinement rounds per solve, HHL without fallback is strict on at most one of the three `sanity` instances per configuration (`tiny_B`, $N = 8$, $\hat{\kappa} \leq 84$, at $n_c = 8$) and forces the IPM to use between 64 and 179 rejected directions per configuration; 12 clock qubits add no success. Regularising the scaled system by δI lowers κ but limits refinement to a residual of about δ, and the block test alone diverges on SDPLIB, so neither is a remedy.

8.3 Polynomial solver: accuracy and degree

Fig. 9(b) shows that the forward error of the polynomial solver follows the requested ε: 2.4×10^{-13} (CKS) and 3.9×10^{-11} (optimal) at $\varepsilon = 10^{-10}$, and 4.7×10^{-10} and 2.0×10^{-7} at $\varepsilon = 10^{-6}$. On the 38 (system, ε) pairs on which both polynomials ran, the optimal polynomial needs 2.70× fewer walk steps (range 2.26–3.36) and has a 1.89× higher success probability, i.e. 4.8× fewer walk steps per successful sample (Table 10, Fig. 10). Eight of the 96 configurations (four per polynomial) exceeded the simulation cap ($\kappa \geq 2 \times 10^4$) and are reported as not run, never as successes.

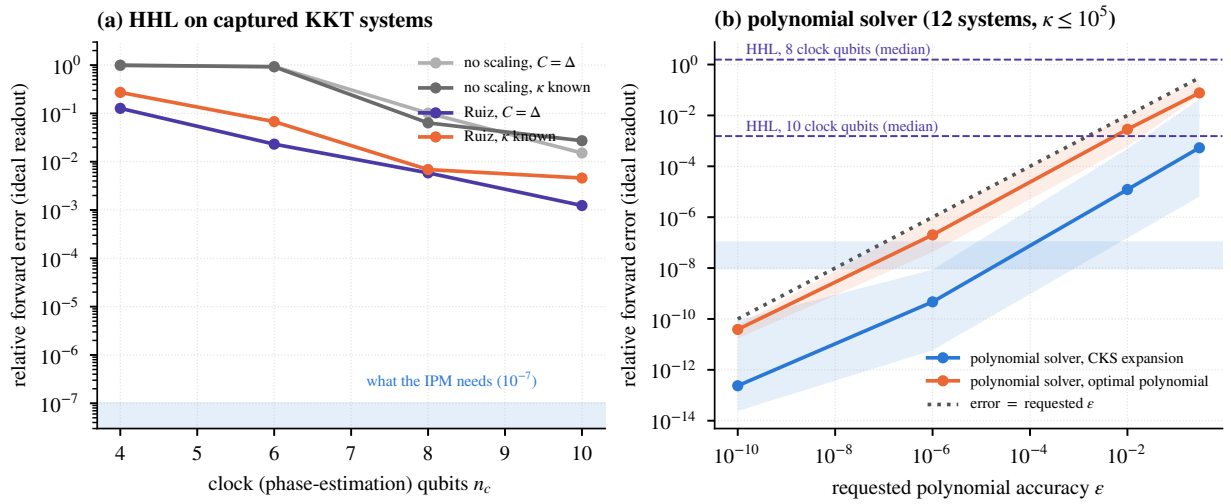


Fig. 9: Forward error of quantum linear solvers on captured KKT systems with ideal (statevector) readout. (a) HHL versus clock-register size for the four scaling and reciprocal variants (medians over 20 systems, session M). (b) Polynomial solver versus requested accuracy ε for the CKS expansion and the optimal polynomial (medians and min–max over 12 systems, session A); the CKS bound over-delivers by 3–4 orders of magnitude, whereas the optimal polynomial sits at the target. The shaded band marks the 10^{-7} requirement of Section 7.6.

8.4 Cost of refinement: precision versus samples

The cost model (11) predicts that the cheapest way to reach a given final residual is a low-precision circuit with few samples per round. Fig. 11 and Table 11 test this prediction on 9 captured systems ($\hat{\kappa} \le 4.3 \times 10^3$) refined to a residual of 10^{-10} with finite-shot readout under emulated amplitude amplification. Every configuration in the table converged on every system (the optimal polynomial at $\varepsilon = 10^{-6}$ exceeded its generation cap on one or two systems and is omitted).

Low precision and few samples are cheapest. Relative to a high-precision baseline—the CKS expansion at $\varepsilon = 10^{-6}$ with 20 000 samples per round—the paired median reduction in block-encoding queries is 20× from using 500 samples per round alone, 237× with the optimal polynomial at $\varepsilon = 0.3$ and 500 samples, and 624× at 200 samples, at the price of 20 instead of 7 refinement rounds.

The contraction rate is set by the larger of two error sources. With the accurate CKS circuit, the contraction per round is set by the sampling error (0.22, 0.15, 0.10, 0.043 and 0.018 at 200, 500, 10^3, 4×10^3 and 2×10^4 samples per round), in agreement with $c_0\sqrt{N/(2S)}$; with the $\varepsilon = 0.3$ circuit it saturates at 0.25–0.30 $\approx \varepsilon$ regardless of the sample count. The readout contribution does not depend measurably on $\hat{\kappa}$ (Remark 5.3), so the conditioning enters the cost through the circuit depth and the post-selection probability, not through the number of rounds.

8.5 Quantum solvers inside the IPM

Table 12 and Fig. 12 give the central quantum result: the IPM runs with a simulated quantum KKT solver and *no* classical fallback, and every direction is subject to the combined acceptance test ($\eta = 0.1$, $\omega \le 10^{-8}$).

Polynomial solver. With ideal readout, the solver is strict on 8/8 instances both at high circuit precision ($\varepsilon = 10^{-10}$, no refinement needed) and at $\varepsilon = 0.3$ with refinement (971 rounds over 90

Table 10: Degree d (walk steps, i.e. block-encoding queries per circuit) and LCU 1-norm $\|c\|_1$ of the two inversion polynomials at equal target accuracy ε on $\mathcal{D}_\kappa$. Cost ratio $= (d_{\mathrm{CKS}}/d_{\mathrm{opt}})\,(\|c\|_1^{\mathrm{CKS}} / \|c\|_1^{\mathrm{opt}})^2$ (walk steps per successful sample).

κ	ε	d_{CKS}	d_{opt}	$\|c\|_1^{\mathrm{CKS}}$	$\|c\|_1^{\mathrm{opt}}$	d ratio	cost ratio
5	0.3	47	15	9.5	7.8	3.13	4.6
5	10^{-2}	85	31	14.1	10.5	2.74	4.9
5	10^{-6}	183	77	22.2	16.0	2.38	4.5
20	0.3	261	85	46	36	3.07	5.1
20	10^{-6}	805	337	93	67	2.39	4.6
100	0.3	1 777	581	272	205	3.06	5.4
100	10^{-6}	4 489	1 843	484	350	2.44	4.7
1 000	0.3	24 501	8 113	3 214	2 379	3.02	5.5
1 000	10^{-6}	51 545	20 725	5 137	3 698	2.49	4.8

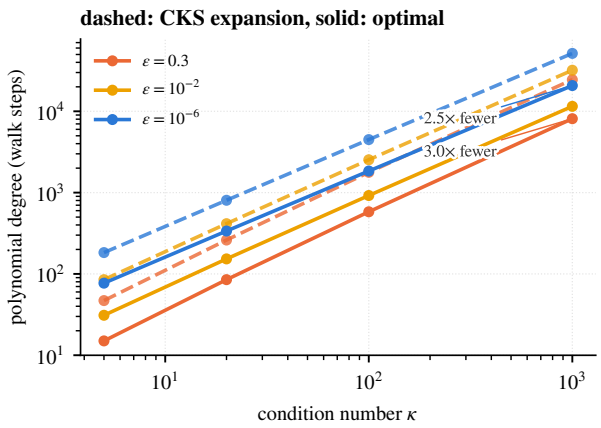


Fig. 10: Degree of the two inversion polynomials versus κ at three target accuracies (Table 10).

solves); CVXOPT declares *optimal* on 7 and 8 of them, the exception being a run whose next IPM iteration exceeded the depth cap after the independent tolerances had been met. With finite-shot readout, it is strict on 6/8 instances at 250 samples per setting (500 per round) and on 7/8 at 10^4. In the three non-strict augmented runs every solve passed the acceptance test, and the run stopped at the depth cap in the eighth IPM iteration ($\kappa \approx 3.8 \times 10^4$, 1.4×10^6 walk steps); Section 8.6 explains why these runs need eight iterations instead of five.

Replication. Sessions A and E give the same strict counts on all eight instances, and sessions C and D, which run seven of them, give 7/7, 7/7, 5/7 and 6/7 for the four polynomial configurations; session D gives 7/7 and 2/7 for the null-space solver of Section 8.7 (Appendix C). Because the quantum samples are seeded, these sessions test the software path on different machines; the sampling variance is measured in Section 8.6.

HHL under the same refinement budget. With 60 refinement rounds per solve, HHL is strict on 2 of the 4 instances tried with 10 clock qubits (ideal readout) and fails on the others after using 50 rejected directions. Iterative refinement converges for any solver with residual factor $\xi < 1$, and HHL has $\xi < 1$ exactly where its clock resolves the spectrum; the polynomial solver satisfies $\xi < 1$ by construction (Theorem 5.2).

Cost. The runs with 500 samples per round used 7.7×10^5 accepted samples and 8.2×10^{12} block-encoding queries in total (with ideal amplitude amplification), and the runs with 2×10^4

Table 11: Cost of refining a finite-shot solve to a residual of 10^{-10} on 9 captured KKT systems (medians, session A). *queries* = rounds × samples × walk steps $/\sqrt{p_{\text{succ}}}$ (ideal amplitude amplification). Reduction: paired median ratio to the high-precision baseline in the first row.

polynomial	ε	samples/round	rounds	walk steps	queries	reduction
CKS	10^{-6}	20 000	7	1 583	1.8×10^{10}	1×
CKS	10^{-6}	500	13	1 583	1.0×10^{9}	20×
CKS	10^{-6}	200	16	1 583	4.9×10^{8}	40×
optimal	10^{-2}	500	12	311	9.2×10^{7}	220×
optimal	0.3	4 000	17	183	5.8×10^{8}	39×
optimal	0.3	500	20	183	8.7×10^{7}	237×
optimal	0.3	200	20	183	3.5×10^{7}	624×

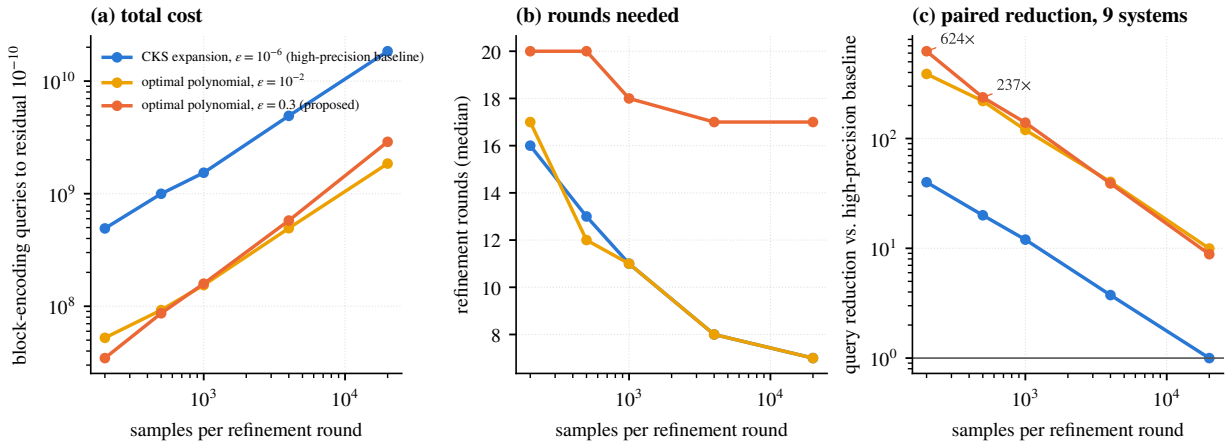


Fig. 11: Cost of refinement to residual 10^{-10} versus samples per round for three (polynomial, ε) settings (9 captured systems, medians, session A). Low circuit precision and few samples per round minimise the total number of block-encoding queries (a) at the expense of more rounds (b); (c) paired reduction relative to the high-precision baseline.

samples per round 1.1×10^7 samples and 1.4×10^{15} queries—for systems of dimension at most 32 that a classical LU decomposition solves in microseconds. The smallest post-selection probability was 3.7×10^{-8}, and the largest circuit needed 29 qubits.

8.6 Sampling seeds and samples per round

Each configuration of Table 12 ran with one sampling seed. Table 13 and Fig. 13 repeat the $\varepsilon = 0.3$ polynomial solver on the same eight instances with five sampling seeds at 100, 250, 1 000 and 4 000 accepted samples per refinement round (both settings together).

Success depends on the instance, not on the draw. Strict success rises from 15/40 to 30/40, 30/40 and 39/40. At 100 and 250 samples, every seed gives the same outcome on every instance; the only seed-to-seed variation occurs at 4 000 samples (0.88–1.00 per seed). At 100 samples the refinement stalls on the five larger instances, and the IPM is forced to use 65 rejected directions; from 250 samples on no rejected direction is ever used, and every failure is a depth-cap stop on the two instances with $\kappa(P) = 10^3$ (`dense_kappa_n10_k3_s0` and `dense_kappa_n10_k3_s1`).

Near-threshold directions lengthen the run. Refinement stops as soon as the combined test holds, so with finite shots the accepted directions have block ratios β of 0.03–0.06, close to $\eta = 0.1$,

Table 12: Quantum KKT solvers inside `coneqp` without classical fallback (session M; sessions A and E give identical strict counts). *strict*: independent final tolerances reached, zero fallbacks, no rejected direction used; *opt.*: runs to which CVXOPT itself assigns status *optimal*. "ideal": statevector readout; "S": accepted samples per setting and round (emulated amplitude amplification). *walk steps*: median polynomial degree per solve.

solver	readout	inst.	strict	opt.	rejected used	refine rounds	walk steps
refinement budget of 4 rounds per solve							
HHL $n_c = 8$ (Ruiz, κ-known)	ideal	3	1	1	179	931	–
HHL $n_c = 10$	ideal	3	0	2	70	458	–
HHL $n_c = 12$	ideal	3	1	2	64	385	–
HHL $n_c = 8$ (fallback on)	$S = 1000$	3	0	3	0 (30 fb)	124	–
refinement budget of 60 rounds per solve							
HHL $n_c = 8$	ideal	3	1	1	46	2 974	–
HHL $n_c = 10$	ideal	4	2	2	50	3 290	–
HHL $n_c = 10$	$S = 250$	3	2	3	1	354	–
LCU augmented, optimal, $\varepsilon = 10^{-10}$	ideal	8	**8**	7	0	0	897
LCU augmented, optimal, $\varepsilon = 0.3$	ideal	8	**8**	8	0	971	162
LCU augmented, optimal, $\varepsilon = 0.3$	$S = 250$	8	6	5	0	1 431	231
LCU augmented, CKS, $\varepsilon = 10^{-6}$	$S = 10\,000$	8	7	6	0	466	1 595
LCU null space, $\varepsilon = 0.3$	ideal	8	**8**	8	0	321	40
LCU null space, $\varepsilon = 0.3$	$S = 250$	8	2	2	2	550	20

Table 13: Polynomial solver ($\varepsilon = 0.3$, optimal polynomial) inside `coneqp` without fallback, 8 instances $\times$ 5 sampling seeds (session M). *per seed*: range over seeds of the strict-success share; *opt.*: runs with CVXOPT status *optimal*; medians per run for IPM iterations, refinement rounds, samples and block-encoding queries (ideal amplitude amplification).

samples/round	strict	per seed	opt.	rejected	IPM its	rounds	samples	queries
100	15/40	0.38	20	65	6.0	646	6.6×10^4	4.1×10^{11}
250	30/40	0.75	20	0	5.5	276	7.2×10^4	3.1×10^{11}
1 000	30/40	0.75	22	0	5.5	158	1.7×10^5	8.7×10^{11}
4 000	39/40	0.88–1.00	32	0	5.0	140	6.1×10^5	1.9×10^{12}

whereas with ideal readout the first solve already overshoots the test (median $\beta \approx 4 \times 10^{-3}$, down to 10^{-4} on the two $\kappa(P) = 10^3$ instances). On those two instances, the near-threshold directions cost the IPM three additional iterations (8 instead of 5), which carry it to a barrier parameter at which the KKT matrix reaches $\kappa \approx 3.8 \times 10^4$ and the polynomial needs 1.4×10^6 walk steps. This is the inexact-Newton price of accepting directions at the forcing threshold, and it explains the finite-shot failures in Table 12. A tighter forcing parameter for the quantum solver, or one refinement round beyond acceptance, is the evident remedy; both trade samples for IPM iterations.

Few samples per round remain the cheapest setting. Per run, 250 samples per round cost a median 7.2×10^4 samples and 3.1×10^{11} block-encoding queries, and 4 000 samples per round 6.1×10^5 samples and 1.9×10^{12} queries: the number of refinement rounds falls only from 276 to 140 while the samples per round grow sixteenfold, as (11) predicts.

8.7 The null-space target system

The null-space system (4) is positive definite, so the polynomial (10) needs about $\sqrt{\kappa}$ rather than κ walk steps. Table 14 and Fig. 14 compare both targets on the same 12 captured KKT systems, and the last two rows of Table 12 run the null-space solver inside `coneqp`.

Early in the run the null space is cheaper; late in the run it is not. At the first captured IPM

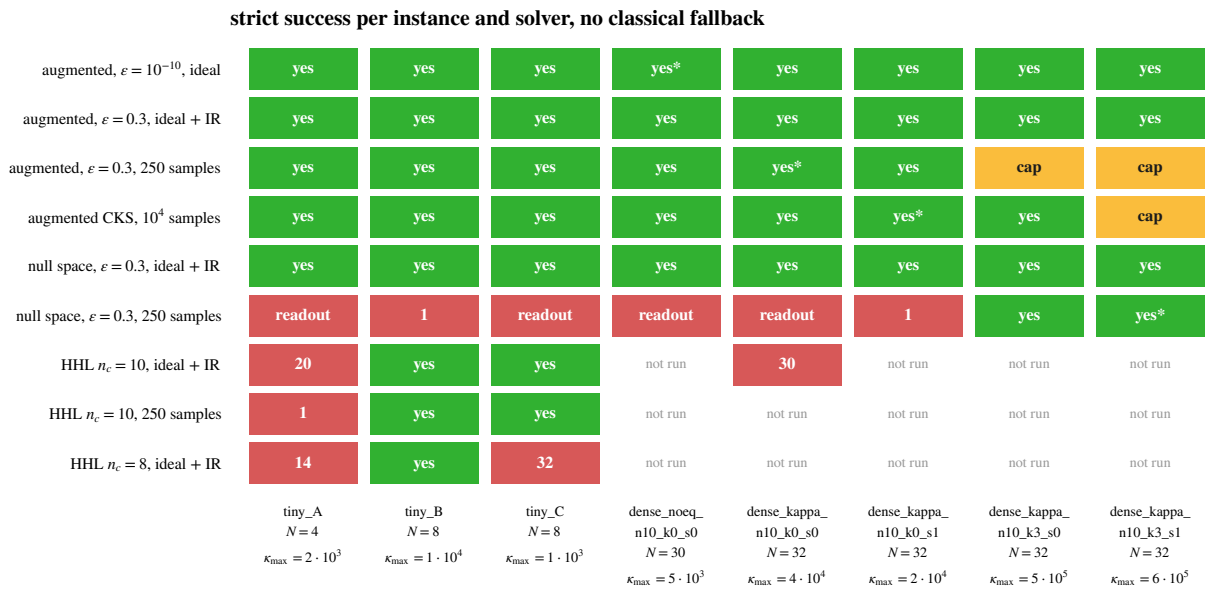


Fig. 12: Per-instance outcome of the no-fallback runs with a refinement budget of 60 rounds per solve (session M). Green: strict success; "yes*": strict, but CVXOPT reports status *unknown* because the depth cap stopped the run after the tolerances had been met. Amber ("cap"): every KKT solve passed the acceptance test, but the run exceeded the simulation's cap of 4×10^5 walk steps before reaching the tolerances. Red: number of rejected directions the IPM was forced to use, or "readout" when the finite-shot reconstruction returned no usable estimate and the run stopped. $\kappa_{\max}$ is the largest condition number of the unscaled KKT matrix encountered in the (strict) ideal-readout polynomial run of that instance.

iteration both targets have about the same condition number (median 19.9 versus 19.4), and the positive definite polynomial needs a median 12× fewer walk steps (2.5–834×). At the last captured iteration, the Jacobi-scaled null-space matrix has $\kappa_{\mathcal{N}} = 4.2 \times 10^5$ (median) against $\hat{\kappa} = 394$ for the equilibrated augmented matrix, because $Z^{\mathsf{T}} S Z$ inherits the μ^{-2} growth of S, and it needs 1.6× more walk steps—as Corollary 5.6 predicts, since $\sqrt{\kappa_{\mathcal{N}}} \approx 650 > 2\hat{\kappa}$ only in the latter case. Over all 12 systems the median saving at $\varepsilon = 0.3$ is 3.0×, and the target is 2–4× smaller ($n - p$ instead of N), which saves up to 11 qubits on the largest system.

With ideal readout the target works inside the IPM. The null-space solver is strict and optimal on 8/8 instances, with a median of 40 walk steps per solve against 162 for the augmented solver on the same KKT matrices (4.0× fewer), 321 instead of 971 refinement rounds, and at most 23 qubits in both cases. Its post-selection probability, however, falls to 4.5×10^{-13} (augmented: 3.8×10^{-8}), because $\|c\|_1 \approx \kappa_{\mathcal{N}}$.

With finite shots it fails. At 250 samples per setting only 2/8 runs are strict: on five instances, the reference-branch reconstruction returns no usable estimate in one measurement setting, which yields a non-finite direction and stops the run, and on one instance the IPM uses a rejected direction. The completion (4) explains the fragility: the multiplier u_y is recovered through P, so that the readout error in u_x is multiplied by $\|P\|$. With ideal readout at $\varepsilon = 0.3$, the forward error of the completed direction is at most 0.14 on the `sanity` instances but 37 on the first captured system of `dense_kappa_n20_k6_s0` ($\kappa(P) = 10^6$; Table 14). Classically, CG on the null-space system needs 2.1× fewer Krylov iterations than augmented MINRES on the dense families but not on the sparse ones (Fig. 14c). The null-space target thus trades circuit depth for success probability and readout robustness: it is attractive early in the run and when P is well conditioned, and a hybrid that

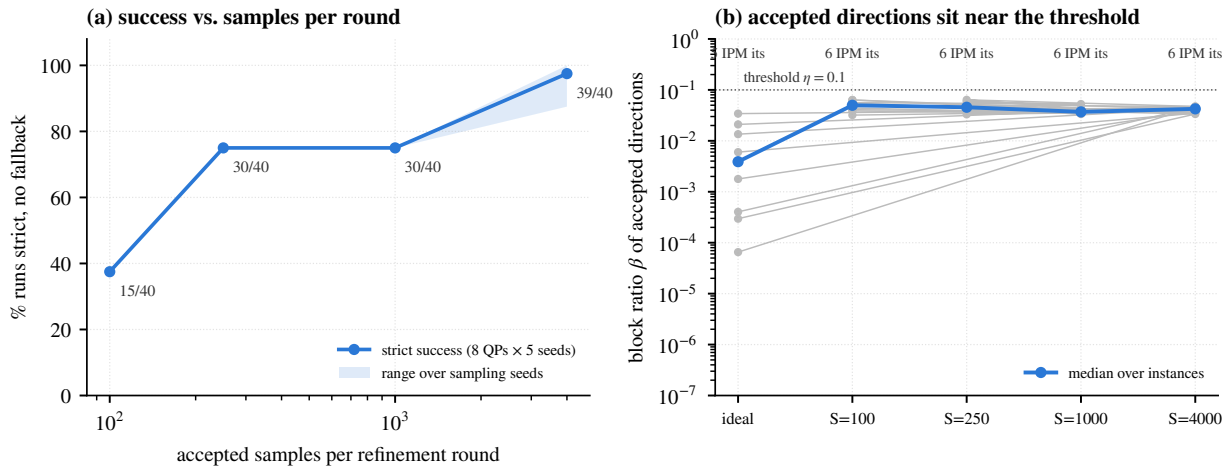


Fig. 13: Finite-shot polynomial solver over sampling seeds (session M). (a) Strict success versus accepted samples per refinement round; band: range over the five seeds. (b) Block ratio β of the accepted directions per instance (grey) and median (blue), with the median number of IPM iterations above each group; dotted: the forcing threshold $\eta = 0.1$.

switches to the augmented target as μ decreases is the natural next step.

8.8 Synthesis: quantum resource reductions and validation of the analysis

Table 16 collects the resource reductions attributable to each methodological component, together with the theoretical prediction of Section 5; Table 15 and Fig. 15 provide the supporting measurements. All reductions are relative to alternative quantum configurations in the same simulation, not to classical solvers.

Equilibration. Across all families, Ruiz equilibration lowers the maximum condition number along the IPM run by a median of 5.4×10^3 on `dense_kappa`, 4.3×10^4 on `sparse_banded`, 3.3×10^4 on `sparse_lowfill` and 7.5×10^4 on `sparse_scale` (Table 15, Fig. 15a). By Corollary 5.5, the corresponding degree of the optimal polynomial at $\varepsilon = 0.3$ falls by a median of 1.2×10^4 to 2.2×10^5; on the unscaled matrices, the median degree would exceed the simulation cap of 4×10^5 walk steps by factors of about 50 (`dense_kappa`) to 400 (`sparse_lowfill`), and it exceeds the cap on every instance of the three sparse families and on 85 % of the `dense_kappa` instances. On well-scaled matrices (`sanity`, `dense_noeq`), the reduction is small (median 1.9–2.2), and on some SDPLIB instances equilibration slightly increases κ (minimum ratio 0.36); the Maros–Mészáros matrices remain ill-conditioned after equilibration (κ up to 4.2×10^{15}), so neither the classical nor the quantum solvers benefit there.

Readout model. Fig. 15(b) confirms the empirical readout model of Remark 5.3: with an accurate circuit, the contraction per round follows $\xi = c_0\sqrt{N/(2S)}$ with $c_0 = 1.10$ over two orders of magnitude, independently of the polynomial and of $\hat{\kappa}$, and with $\varepsilon = 0.3$ it saturates at the polynomial error. The worst-case factor $\hat{\kappa}$ in (12) is not observed.

Classical work per solve. Fig. 15(c) compares the classical matrix–vector products per KKT solve: 10–19 refinement rounds for the refined LCU solver at $N \leq 32$, against 24–27 MINRES iterations at the same sizes and up to 2 061 at $N = 79\,360$. Under the empirical readout model with $S \propto N$, Theorem 5.2 predicts a number of rounds independent of N (shaded band), whereas the MINRES count grows with N. The ratio k/R, which bounds any advantage by Proposition 5.7, is at most 2.5 at the simulated sizes.

Table 14: The same captured KKT systems as augmented (Ruiz-equilibrated, indefinite, odd optimal polynomial) and null-space (Jacobi-scaled, positive definite, shifted Chebyshev polynomial) targets at $\varepsilon = 0.3$ (session M). *stage*: first, middle or last captured IPM iteration; N/dim: KKT and null-space dimension; d: walk steps; p_{succ} and forward error of the completed direction for the null-space target with ideal readout.

instance	stage	N/dim	$\hat{\kappa}$ aug.	$\kappa_{\mathcal{N}}$ null	d aug.	d null	ratio	p_{succ} null	fwd. err.
`dense_kappa_n20_k6_s0`	first	64/16	9.9×10^{4}	6.4×10^{4}	1 330 217	1 595	834	3.0×10^{-9}	3.7×10^{1}
`dense_kappa_n20_k6_s0`	middle	64/16	4.3×10^{3}	3.1×10^{3}	42 993	264	163	4.0×10^{-4}	1.2×10^{-2}
`dense_kappa_n20_k6_s0`	last	64/16	3.0×10^{2}	3.1×10^{2}	2 211	62	35.7	1.2×10^{-2}	9.5×10^{-3}
`tiny_A`	first	4/2	1.9	3.6	5	2	2.50	5.7×10^{-1}	1.4×10^{-1}
`tiny_A`	middle	4/2	1.3×10^{1}	3.9×10^{2}	53	72	0.74	4.5×10^{-1}	1.6×10^{-3}
`tiny_A`	last	4/2	1.3×10^{3}	3.9×10^{6}	11 035	16 517	0.67	4.5×10^{-1}	7.3×10^{-6}
`tiny_B`	first	8/2	4.0	1.5	11	1	11.0	7.6×10^{-1}	1.1×10^{-2}
`tiny_B`	middle	8/2	5.4	9.9	17	5	3.40	8.1×10^{-3}	1.2×10^{-3}
`tiny_B`	last	8/2	8.4×10^{1}	5.2×10^{4}	503	1 413	0.36	3.4×10^{-10}	2.0×10^{-7}
`tiny_C`	first	8/4	3.6×10^{1}	3.5×10^{1}	183	14	13.1	2.4×10^{-3}	1.3×10^{-2}
`tiny_C`	middle	8/4	4.5×10^{1}	9.0×10^{2}	243	124	1.96	1.4×10^{-3}	1.5×10^{-3}
`tiny_C`	last	8/4	4.9×10^{2}	7.9×10^{5}	3 803	6 744	0.56	1.6×10^{-6}	1.4×10^{-4}

Table 15: Effect of per-iteration Ruiz equilibration on the KKT condition number: per instance the maximum along the IPM run, then the median over the instances of a family (session M; sparse families for $N \leq 620$, where κ was computed; SDPLIB: session A; Maros–Mészáros: offline diagnostic). *degree red.*: median reduction of the optimal-polynomial degree at $\varepsilon = 0.3$ by (9) (Corollary 5.5).

family	inst.	N	$\kappa(\tilde{K})$	$\kappa(D\tilde{K}D)$	reduction	range	degree red.
`sanity` (`tiny_A`–`tiny_C`)	3	4–8	2.4×10^{3}	4.9×10^{2}	2.2	2.0–1.3×10^{2}	2.4
`dense_kappa`	60	32–256	1.3×10^{6}	2.4×10^{3}	5.4×10^{3}	7.6–6.3×10^{5}	1.2×10^{4}
`dense_noeq`	8	30–120	5.0×10^{5}	3.9×10^{4}	1.9	8.0×10^{-1}–1.4×10^{1}	2.0
`sparse_scale`	4	77–620	7.3×10^{6}	4.3×10^{2}	7.5×10^{4}	2.4×10^{2}–5.5×10^{5}	2.2×10^{5}
`sparse_banded`	10	310–620	4.8×10^{6}	1.1×10^{2}	4.3×10^{4}	9.3×10^{3}–4.3×10^{5}	1.2×10^{5}
`sparse_lowfill`	6	310–620	9.6×10^{6}	3.2×10^{2}	3.3×10^{4}	1.8×10^{4}–8.3×10^{4}	8.0×10^{4}
SDPLIB (`conelp`)	9	25–487	2.4×10^{11}	2.7×10^{9}	1.2×10^{3}	3.6×10^{-1}–4.6×10^{6}	5.2×10^{3}
Maros–Mészáros (diagnosed)	8	256–1 565	1.4×10^{14}	4.4×10^{7}	9.1×10^{4}	9.0×10^{-1}–1.3×10^{11}	1.4×10^{5}

8.9 Comparison with published numerical studies

Table 17 places these results next to the numerical demonstrations we are aware of. Values in the rows marked † are taken from the cited papers and were not reproduced by us.

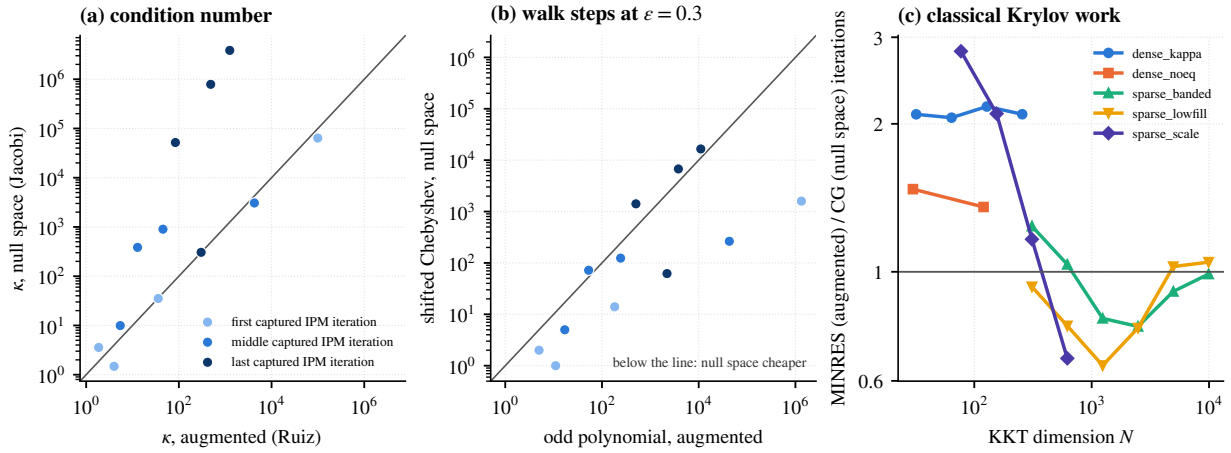


Fig. 14: Augmented versus null-space target (session M). (a) Condition numbers and (b) walk steps at $\varepsilon = 0.3$ on 12 captured systems, shaded by IPM progress; below the diagonal the null-space target is cheaper. (c) Classical Krylov work, MINRES on the augmented system over CG on the null-space system, per family and size.

Table 16: Quantum resource reductions attributable to each methodological component (simulation; relative to alternative quantum configurations, not to classical solvers).

component	resource	theory	measured reduction	source
per-iteration Ruiz equilibration	κ in d and $\|c\|_1$	$d \propto \kappa \log(\kappa/\varepsilon)$, $\|c\|_1 = \tilde{\Theta}(\kappa)$	κ: 4.3×10^4 (`sparse_banded`), 5.4×10^3 (`dense_kappa`); d: 1.2×10^5, 1.2×10^4	Table 15
optimal vs. CKS polynomial	d, $\|c\|_1$	same order, smaller constant	walk steps 2.70×; per accepted sample 4.8× (38 pairs)	Sec. 8.3
low-precision circuit ($\varepsilon = 0.3$) + refinement vs. high-precision circuit	samples, rounds, d	Eq. (11)	queries 237× (500 samples/round), 624× (200)	Sec. 8.4
refinement vs. direct readout to $\varepsilon_f = 10^{-8}$	samples per KKT solve	$\Theta(N/\varepsilon_f^2) \to \Theta(N \log(1/\varepsilon_f))$	3.9×10^{17} (model, $c_0 = 1.10$, $N = 32$) vs. 7.3×10^3 (measured)	Cor. 5.4
positive definite null-space target	d	$\sqrt{\kappa}$ instead of κ	walk steps 4.0× inside the IPM; p_{succ} 4.5×10^{-13} vs. 3.8×10^{-8}	Sec. 8.7

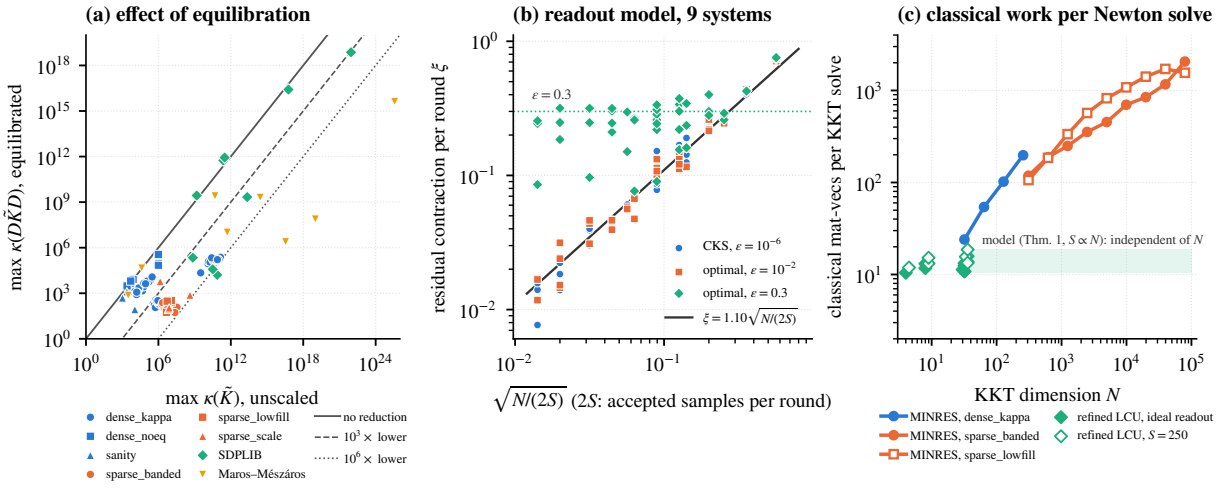


Fig. 15: Quantum resource reductions. (a) Maximum condition number along the IPM run before and after Ruiz equilibration, per instance and family; lines mark no, $10^3\times$ and $10^6\times$ reduction. (b) Residual contraction per refinement round versus $\sqrt{N/(2S)}$ on 9 captured systems for three circuit settings, with the fitted readout model $\xi = c_0\sqrt{N/(2S)}$ (Remark 5.3). (c) Classical matrix–vector products per KKT solve: MINRES iterations on three families against refinement rounds of the LCU solver with ideal and finite-shot readout; the band is the prediction of Theorem 5.2 with $S \propto N$.

Table 17: Numerical studies of quantum linear solvers for optimisation. Values marked † are reported by the cited authors.

study	setting	result
Mohammadisiahroudi et al. [43]†	quantum IPM for LP; Qiskit HHL simulator; linear systems of dimension ≤ 16	inner and outer iterative refinement improve the precision from $\approx 10^{-1}$ to $\approx 10^{-4}$
Harkness et al. [33]†	standalone linear systems; HHL with 3–5 clock qubits; dimension 8, $\kappa = 5$ (simulator and IBM Heron); dimension 64 with 100–1000 shots	residual $\leq 10^{-9}$ in 9–10 rounds noiseless; 0.86 $\rightarrow$ 0.07–0.10 on hardware; 10^{-3} in ≈ 10 rounds with 100 shots
Dalzell et al. [19]†	resource estimate (no simulation) of a quantum IPM for a 100-asset portfolio SOCP	$\approx 8\times10^6$ logical qubits and $\approx 7\times10^{29}$ T gates; tomography and κ dominate
Ozbaygin et al. [46]†	VQE as separation oracle in an SDP cutting-plane scheme; SDPLIB `control1`–`control10`; simulator, noise model, IBM hardware	valid cuts throughout; 32–82 % of the initial gap closed versus 75–82 % with exact eigendecomposition; oracle 10^3–$10^4\times$ slower; matrices ≥ 98 % dense in the Pauli basis
this work	convex QP inside unmodified CVXOPT; LCU polynomial solver at the walk level with low-precision refinement; KKT dimension ≤ 32 inside the IPM, $\kappa \leq 3.8\times10^4$; strict no-fallback accounting; six sessions	8/8 strict with ideal readout; 38–98 % with 100–4 000 samples per round over 5 seeds; 237× fewer queries than a high-precision circuit; 8.2×10^{12} queries at 500 samples per round; HHL 2/4; null-space target 4.0× shallower, 2/8 with finite shots

9 Discussion

9.1 Significance of the methodological contributions

The study examined, inside an unmodified production IPM and under one acceptance test, one fallback protocol and one accounting, what accuracy a Newton direction must have, how it can be certified, and which classical and quantum solvers deliver it. Five findings are of general significance.

Equilibration, not the acceptance test, makes an unpreconditioned Krylov solver reliable on regular problems. Every Ruiz-equilibrated variant with a tolerance of 10^{-10} or tighter is fallback-free on 100 % of the synthetic runs, whereas every unscaled variant needs fallback in 38–50 % of them. The equilibrated augmented matrix has $\kappa \in [80, 180]$ along a run in which the NT scaling spreads over six orders of magnitude—the measured form of the boundedness theory for unreduced KKT systems. On the Maros–Mészáros QPs, which violate the assumptions of that theory, the same matrix reaches $\kappa = 4.2 \times 10^{15}$ and MINRES needs fallback on more than half of the instances.

Fill-in and regularity, not the method, decide whether the iterative solver is faster. Under high fill-in the crossover is at $N \approx 5 \times 10^3$ and the advantage reaches 9.4× at $N \approx 2 \times 10^4$, with less than ±30 % spread over instances; under low fill-in there is no crossover below $N \approx 4 \times 10^4$; and on 48 real-world QPs the solver's own factorisation is fastest on 40. Latency, not arithmetic, sets the useful range of a GPU Krylov kernel: the mat-vec is 13.4× faster on the T4 at $N = 79\,360$, but a baseline MINRES iteration costs a constant 0.7 ms; one synchronisation per iteration and fused updates recover 1.14–1.29×, and the GPU pays off from $N \approx 2 \times 10^4$.

A Newton direction needs a relative forward error between $\tau/10$ and τ. This precision law, measured at every tolerance from 10^{-8} to 10^{-4}, is the accuracy target for any inexact solver. A backward-error certificate alone is insufficient when $\kappa(\tilde{K})$ is large (Lemma 4.1); the per-block forcing test supplies the IPM-specific structure and binds on the equality block late in the run, which is where the 22–34 % over-solving of the combined test originates and what an IPM-aware stopping rule [63] could recover.

Low-precision refinement is the decisive design choice for a quantum KKT solver. Inside the linear solve, refinement converges for any solver with $\xi < 1$, which the polynomial solver guarantees by construction and HHL achieves only where its clock resolves the spectrum. Operating the circuit at $\varepsilon = 0.3$ with a few hundred samples per round is 237–624× cheaper in queries than a high-precision circuit, keeps the circuit depth independent of the IPM tolerance (Corollary 5.4), and replaces the $\mathcal{O}(N/\tau^2)$ samples of direct readout by $\mathcal{O}(N \log(1/\tau))$—$3.9 \times 10^{17}$ versus 7.3×10^3 samples per solve at the default tolerance.

The forcing threshold interacts with the cost of a quantum solver. With finite shots, refinement stops at directions whose block ratio is close to η; such directions add IPM iterations, and the additional iterations reach barrier parameters at which the KKT matrix, and hence the circuit depth, grows by an order of magnitude. A quantum solver inside an IPM should therefore be run with a tighter forcing parameter than a classical one, and its cost model must include the effect of η on the number of IPM iterations.

9.2 Outlook for a quantum advantage

The results do not demonstrate a quantum advantage, and the analysis explains why none is possible at the simulated sizes: for $N \leq 32$ the ratio k/R is at most 2.5, and the query counts exceed the classical cost by many orders of magnitude. The analysis also identifies the regime in which the methodology could yield one. Proposition 5.7 requires dense, well-equilibrated KKT systems—large $\bar{\nu}$ and moderate $\hat{\kappa}$—solved to IPM accuracy by low-precision refined circuits with $S = \Theta(N)$ samples, on hardware whose block-encoding queries do not grow with $\mathrm{nnz}(\tilde{K})$, and

preferably with an optimal-scaling QLSA ($\gamma = 1$). In that regime the advantage is polynomial, bounded by k/R, and grows with the classical iteration count, which on `dense_kappa` increases from 24 to 198 between $N = 32$ and $N = 256$ while R does not. Each methodological component moves the cost towards this regime: equilibration lowers $\hat{\kappa}$, which enters as $\hat{\kappa}^\gamma$, by three to five orders of magnitude; low-precision refinement removes the $1/\varepsilon_f^2$ sample dependence and fixes the depth; the optimal polynomial reduces the queries per accepted sample by 4.8×; and the null-space target reduces the depth by the factor $2\hat{\kappa}/\sqrt{\kappa_{\mathcal{N}}}$ where Corollary 5.6 applies. Because these components are classical pre- and post-processing steps, they apply unchanged to other QLSA-based IPMs and to optimal-scaling solvers [17, 18], which would remove the post-selection factor $\|c\|_1$; the coherent tomography of [58] would improve the sample dependence further. As in the VQE cutting-plane study [46], the readout of a classical vector, not the preparation of the solution state, remains the dominant obstacle that a hardware realisation would have to overcome.

9.3 Limitations

The quantum results are classical simulations of ideal circuits that assume free block encoding, state preparation, knowledge of κ and ideal amplitude amplification; the KKT systems inside the IPM have dimension at most 32, and success counts beyond $\kappa \approx 4 \times 10^4$ are limited by the simulation's depth cap rather than by the algorithm. The empirical readout model is fitted on systems with $N \leq 64$, and the instantiation of Proposition 5.7 at larger N is an extrapolation under stated assumptions. The Maros–Mészáros subset stops at $N \approx 10^4$ and excludes instances on which `coneqp` itself fails; the condition-number diagnostic covers eight of its instances. The claims do not extend to SDP (Appendix B), and we have not compared against GPU direct solvers [14] or first-order GPU solvers [38]. The largest synthetic sizes rest on two instances (`sparse_banded`) or one (`sparse_lowfill`), replicated in further sessions. Timings vary by up to 1.3× in the median between CPU-only and GPU sessions.

9.4 Directions for future research

On the classical side, the real-world results call for a preconditioner or regularisation that restores bounded conditioning on degenerate problems—primal–dual regularisation as in interior-point proximal methods [50] combined with general-purpose preconditioners [9, 30] is the natural candidate, and the exact-deflation bound of Section 7.2 (3.7–6.2×) is the gap that a better source of approximate eigenvectors than Lanczos harvesting [39] would have to close. On the quantum side, the forcing parameter should be chosen jointly with the sample count; a hybrid target—null space early in the run, augmented late, switched by the criterion of Corollary 5.6—would combine the $\sqrt{\kappa}$ degree of the first with the robustness of the second; and testing the readout model and the conditions of Proposition 5.7 on larger dense systems, with resource estimates for concrete block encodings, would determine whether the advantage regime identified here is reachable.

10 Conclusion

We have presented a certified framework for replacing the KKT factorisation of a production interior-point solver by classical Krylov or simulated quantum linear solvers, and four methodological components that make such replacements reliable and economical: per-iteration symmetric equilibration, a combined block-forcing and backward-error acceptance test, a latency-optimised GPU MINRES kernel, and a low-precision refined polynomial quantum solver with an optional positive definite null-space target. On the classical side, equilibrated MINRES is fallback-free on

all synthetic runs, up to 9.4× faster than sparse $\mathrm{LDL}^{\mathsf{T}}$ under high fill-in and 3.75× faster on the GPU at $N = 79\,360$, while low fill-in and degenerate real-world QPs mark the limits of these gains. A Newton direction needs a relative error between $\tau/10$ and τ. On the quantum side, we proved contraction, round and query bounds for the refined solver and derived how precision, equilibration and the choice of target enter them; in simulation, these components reduce the block-encoding queries per solve by 237×, the circuit depth by 2.70× and a further 4.0×, and the samples per solve by more than thirteen orders of magnitude relative to direct readout, and the IPM converges on quantum-produced directions alone on 8/8 instances with ideal readout and on 39/40 seeded runs with finite sampling. Proposition 5.7 shows that these reductions could translate into a polynomial advantage over classical MINRES only for dense, well-equilibrated systems at sizes far beyond those simulated, and only on hardware with inexpensive block encodings. All quantum results are classical simulations of ideal circuits; no end-to-end quantum speed-up is demonstrated or claimed.

A Reproducibility

All results are produced by a single Jupyter notebook in two builds (CPU and GPU), which runs on Kaggle with the problem-size profile selected in its configuration cell (`smoke`, `small`, `medium` or `large`). The notebook installs Qiskit for the cross-check, downloads the Maros–Mészáros files when Internet access is enabled, records software versions, hardware and seeds in `environment.json` and `config.json` (with a configuration hash), and writes every table of this paper as CSV—among them `benchmark_results.csv`, `linear_solve_logs.csv`, `quantum_results.csv`, `qls_comparison_hhl_vs_cks.csv`, `oss_target_comparison.csv`, `performance_profiles.csv`, `maros_meszaros_instances.csv`, `outer_refinement.csv`, `gpu_matvec_benchmark.csv`, `failures.csv` and `experiment_status.csv`; peak memory is a column of `benchmark_results.csv`—together with the captured KKT systems and an automatically generated report, and compresses them into one result bundle per session. The tables of this paper are extracted from the bundles by `extract_tables_main.py` (sessions M and B–E) and `extract_tables_sessionA.py` (session A); the LaTeX bodies of the result tables are generated from the extracted CSV files by `make_tex_tables.py`, so that no measured number in them is copied by hand (Table 9 is a typed summary that also draws on exploratory runs on the same instances), and the figures are generated by `make_figures.py`. The condition-number diagnostic of Fig. 7(b) is computed by `mm_kappa_diag.py`, which runs the notebook's own solver code on eight Maros–Mészáros instances. Random seeds are derived from a global seed (20260911) and the instance and method names, so that every instance and every quantum sample is reproducible.

B SDPLIB

Ten SDPLIB instances were solved with `conelp` under the same protocol (Table 18, session A). In this session the inexact methods were run on instances with $N \leq 350$, i.e. on the seven instances `truss1`, `truss4`, `hinf1`, `hinf2`, `control1`, `truss3` and `control2`; `truss2`, `qap5` and `theta1` were solved only by the direct solvers, and `theta1` exceeds the size limit of the reference LU decomposition. The combined test is accepted on 4 of the 7 instances, with 1 048 fallbacks in total; the tightest backward-error test (10^{-12}) on 6 of 7, with 361 fallbacks; and the block test alone on none, with 1 483 fallbacks. The equilibrated KKT matrices reach condition numbers of 2.7×10^{9} (`control1`) to 1.8×10^{19} (`hinf1`). The claims of this paper are therefore restricted to QP.

Table 18: SDPLIB per instance (`conelp`, session A). *its*: IPM iterations of the unmodified solver; *acc.*: final tolerance reached (fallback allowed); *fb*: total classical fallbacks of the MINRES variant; $\kappa(D\tilde{K}D)$: largest equilibrated condition number along the exact-LU run. "–": not run.

			LDL^{T}	MINRES, combined		MINRES, backward 10^{-12}		block $\eta = 0.1$	
instance	N	its	acc.	acc.	fb	acc.	fb	fb	$\kappa(D\tilde{K}D)$
`truss1`	25	10	yes	yes	0	yes	0	85	1.6×10^{4}
`truss4`	49	11	yes	no	387	yes	5	138	3.8×10^{4}
`hinf1`	54	100	no	no	165	no	205	17	1.8×10^{19}
`hinf2`	64	22	yes	no	416	yes	58	334	1.4×10^{17}
`control1`	91	26	yes	yes	26	yes	32	341	2.7×10^{9}
`truss3`	118	12	yes	yes	10	yes	12	202	2.2×10^{5}
`control2`	331	27	yes	yes	44	yes	49	366	5.5×10^{11}
`truss2`	389	16	yes	–	–	–	–	–	2.1×10^{9}
`qap5`	487	8	yes	–	–	–	–	–	8.6×10^{11}
`theta1`	1 379	13	yes	–	–	–	–	–	–

C Replication across sessions

Sessions B (GPU, medium profile, 21 September 2026), C (GPU, small profile, 24 September), D (CPU, small profile, 26 September) and E (CPU, medium profile, 27 September) were executed on different Kaggle machines from the main session M. All six sessions use the same instance generator and seeds, so the synthetic instances they share are identical (session B: seeds 0 and 1; sessions C and D: seed 0; session D runs the CPU methods up to $N = 9\,920$; session E runs the complete experiment set of M with the GPU methods omitted). In every session, all classical methods were accepted and strict on the synthetic sparse families with zero fallbacks and with the IPM iteration counts of session M (in session E on all 550 runs).

Table 19 gives, per method and family, the median and the 10–90 % range over (instance, size) pairs of the ratio of a session's whole-run time to that of session M. The two GPU sessions agree with M to within 0.96–1.10 in the median. The CPU-only sessions run the direct solvers 6–31 % slower, and session E also runs MINRES 12–16 % slower, which is why all speed-ups in the paper are computed within one session. Table 20 lists the quantities on which the conclusions rest for every session, and Fig. 16 plots the per-size ratios.

Table 19: Whole-run wall time of a replication session divided by that of session M, per method: median (10–90 % range) over the instances and sizes both sessions ran.

method	B: GPU, medium	C: GPU, small	D: CPU, small	E: CPU, medium
`sparse_banded`				
$\mathrm{LDL}^{\mathsf{T}}$ (CVXOPT)	0.98 (0.74–1.13)	1.02 (0.80–1.16)	1.20 (0.87–1.47)	1.30 (1.09–1.38)
reduced, CHOLMOD	0.99 (0.91–1.05)	0.99 (0.82–1.05)	1.13 (0.90–1.53)	1.17 (1.04–1.40)
lazy refactorisation	0.96 (0.93–1.01)	0.99 (0.92–1.06)	1.06 (0.88–1.26)	1.09 (1.01–1.33)
MINRES, CPU	0.99 (0.89–1.06)	1.07 (0.95–1.12)	1.01 (0.87–1.11)	1.12 (1.06–1.22)
MINRES, GPU	1.02 (0.98–1.04)	1.03 (1.01–1.08)	–	–
MINRES, CPU, baseline kernel	0.97 (0.93–1.01)	1.06 (0.91–1.08)	1.00 (0.86–1.06)	1.12 (1.04–1.19)
MINRES, GPU, baseline kernel	1.01 (0.98–1.05)	1.04 (0.99–1.07)	–	–
`sparse_lowfill`				
$\mathrm{LDL}^{\mathsf{T}}$ (CVXOPT)	0.98 (0.84–1.03)	1.04 (0.91–1.07)	1.15 (1.04–1.52)	1.31 (1.20–1.42)
reduced, CHOLMOD	0.98 (0.91–1.00)	1.05 (0.97–1.08)	1.19 (1.09–1.25)	1.21 (1.08–1.30)
lazy refactorisation	0.97 (0.94–1.00)	1.05 (0.95–1.11)	1.06 (1.01–1.14)	1.11 (1.08–1.19)
MINRES, CPU	0.97 (0.94–1.01)	1.10 (1.01–1.15)	1.02 (0.97–1.08)	1.15 (1.08–1.24)
MINRES, GPU	1.01 (0.99–1.03)	1.04 (1.01–1.08)	–	–
MINRES, CPU, baseline kernel	1.00 (0.93–1.02)	1.09 (0.94–1.17)	1.02 (0.99–1.10)	1.16 (1.08–1.26)
MINRES, GPU, baseline kernel	1.03 (1.00–1.04)	1.06 (1.04–1.08)	–	–

Table 20: Crossover quantities, quantum success counts and further replicated quantities in each session (speed-ups are medians of per-instance paired ratios within the session; "–": not run in that session; session D runs 7 of the 8 quantum instances and 13 Maros–Mészáros instances).

quantity	M	B	C	D	E
MINRES CPU / $\mathrm{LDL}^{\mathsf{T}}$, `sparse_banded`, $N = 4\,960$	1.26	1.28	1.05	1.29	1.47
MINRES CPU / $\mathrm{LDL}^{\mathsf{T}}$, `sparse_banded`, $N = 9\,920$	3.50	3.86	3.12	3.97	4.11
MINRES CPU / $\mathrm{LDL}^{\mathsf{T}}$, `sparse_banded`, $N = 19\,840$	9.42	9.14	9.49	–	9.08
GPU / CPU MINRES, `sparse_banded`, $N = 19\,840$	1.02	1.01	1.10	–	–
GPU / CPU MINRES, `sparse_banded`, $N = 79\,360$	3.75	3.68	3.68	–	–
CHOLMOD / $\mathrm{LDL}^{\mathsf{T}}$, `sparse_lowfill`, $N = 39\,680$	1.83	1.78	1.78	–	–
MINRES GPU / $\mathrm{LDL}^{\mathsf{T}}$, `sparse_lowfill`, $N = 79\,360$	3.85	3.87	3.99	–	–
strict: LCU $\varepsilon = 10^{-10}$, ideal	8/8	8/8	7/7	7/7	8/8
strict: LCU $\varepsilon = 0.3$, ideal	8/8	8/8	7/7	7/7	8/8
strict: LCU $\varepsilon = 0.3$, $S = 250$	6/8	6/8	5/7	5/7	6/8
strict: LCU CKS, $S = 10^4$	7/8	7/8	6/7	6/7	7/8
strict: null space, ideal	8/8	–	–	7/7	8/8
strict: null space, $S = 250$	2/8	–	–	2/7	2/8
Maros–Mészáros: $\mathrm{LDL}^{\mathsf{T}}$ fastest	40/48	–	–	12/13	39/48
Maros–Mészáros: MINRES with fallback	26/48	–	–	5/13	26/48
largest error tolerated on all instances, $\tau = 10^{-6}$	10^{-7}	–	–	10^{-7}	10^{-7}
largest error tolerated on all instances, $\tau = 10^{-4}$	10^{-5}	–	–	10^{-5}	10^{-5}
shot-seed sweep, strict at 250 samples/round	30/40	–	–	25/35	30/40
shot-seed sweep, strict at 4 000 samples/round	39/40	–	–	32/35	37/40

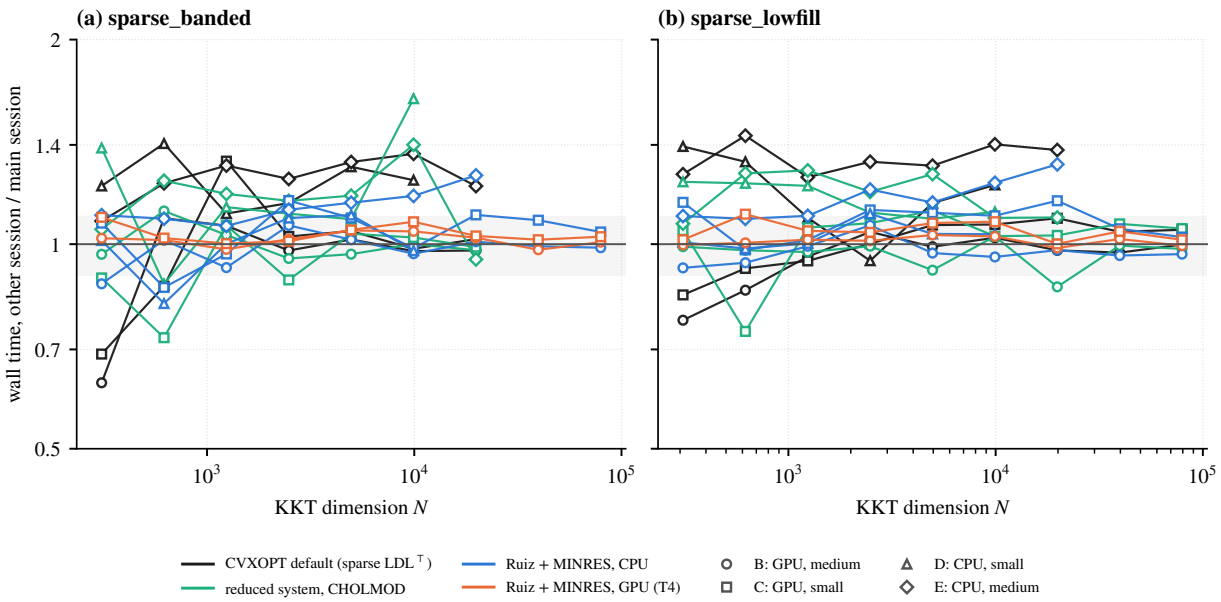


Fig. 16: Ratio of the whole-run wall time in sessions B–E to that in session M, per method and size; the band marks ±10 %.

References

[1] Scott Aaronson. Read the fine print. *Nature Physics*, 11:291–293, 2015.

[2] Ghussoun Al-Jeiroudi and Jacek Gondzio. Convergence analysis of the inexact infeasible interior-point method for linear optimization. *Journal of Optimization Theory and Applications*, 141:231–247, 2009.

[3] Martin S. Andersen, Joachim Dahl, Zhang Liu, and Lieven Vandenberghe. Interior-point methods for large-scale cone programming. *Optimization for Machine Learning*, pages 55–83, 2011.

[4] Martin S. Andersen, Joachim Dahl, and Lieven Vandenberghe. CVXOPT: A Python package for convex optimization. `https://cvxopt.org`, 2023. version 1.3.

[5] Brandon Augustino, Giacomo Nannicini, Tamás Terlaky, and Luis F. Zuluaga. Quantum interior point methods for semidefinite optimization. *Quantum*, 7:1110, 2023.

[6] Stefania Bellavia. Inexact interior-point method. *Journal of Optimization Theory and Applications*, 96(1):109–121, 1998.

[7] Stefania Bellavia, Valentina De Simone, Daniela di Serafino, and Benedetta Morini. Updating constraint preconditioners for KKT systems in quadratic programming via low-rank corrections. *SIAM Journal on Optimization*, 25(3):1787–1808, 2015.

[8] Michele Benzi, Gene H. Golub, and Jörg Liesen. Numerical solution of saddle point problems. *Acta Numerica*, 14:1–137, 2005.

[9] Luca Bergamaschi, Jacek Gondzio, and Giovanni Zilli. Preconditioning indefinite systems in interior point methods for optimization. *Computational Optimization and Applications*, 28: 149–171, 2004.

[10] Brian Borchers. SDPLIB 1.2, a library of semidefinite programming test problems. *Optimization Methods and Software*, 11(1-4):683–690, 1999.

[11] Daan Camps, Lin Lin, Roel Van Beeumen, and Chao Yang. Explicit quantum circuits for block encodings of certain sparse matrices. *SIAM Journal on Matrix Analysis and Applications*, 45 (1):801–827, 2024.

[12] Stéphane Caron and contributors. Maros–mészáros test set for `qpbenchmark` (`.mat` files). `https://github.com/qpsolvers/maros_meszaros_qpbenchmark`, 2026. accessed 26 September 2026.

[13] Erin Carson and Nicholas J. Higham. Accelerating the solution of linear systems by iterative refinement in three precisions. *SIAM Journal on Scientific Computing*, 40(2):A817–A847, 2018.

[14] Yuwen Chen, Danny Tse, Parth Nobel, Paul Goulart, and Stephen Boyd. CuClarabel: GPU acceleration for a conic optimization solver. *ACM Transactions on Mathematical Software*, 2025. arXiv:2412.19027.

[15] Andrew M. Childs, Robin Kothari, and Rolando D. Somma. Quantum algorithm for systems of linear equations with exponentially improved dependence on precision. *SIAM Journal on Computing*, 46(6):1920–1950, 2017.

[16] Shipu Chu, Zachary Santos, and Madeleine Udell. Randomized Nyström preconditioned interior point-proximal method of multipliers. *SIAM Journal on Scientific Computing*, 48(1), 2026. arXiv:2404.14524.

[17] Pedro C. S. Costa, Dong An, Yuval R. Sanders, Yuan Su, Ryan Babbush, and Dominic W. Berry. Optimal scaling quantum linear-systems solver via discrete adiabatic theorem. *PRX Quantum*, 3:040303, 2022.

[18] Alexander M. Dalzell. A shortcut to an optimal quantum linear system solver. arXiv:2406.12086, 2024.

[19] Alexander M. Dalzell, B. David Clader, Grant Salton, Mario Berta, Cedric Yen-Yu Lin, David A. Bader, Nikitas Stamatopoulos, Martin J. A. Schuetz, Fernando G. S. L. Brandão, Helmut G. Katzgraber, and William J. Zeng. End-to-end resource analysis for quantum interior-point methods and portfolio optimization. *PRX Quantum*, 4:040325, 2023.

[20] Gregory Dexter, Agniva Chowdhury, Haim Avron, and Petros Drineas. On the convergence of inexact predictor-corrector methods for linear programming. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.

[21] Elizabeth D. Dolan and Jorge J. Moré. Benchmarking optimization software with performance profiles. *Mathematical Programming*, 91(2):201–213, 2002.

[22] Anders Forsgren, Philip E. Gill, and Joseph R. Shinnerl. Stability of symmetric ill-conditioned systems arising in interior methods for constrained optimization. *SIAM Journal on Matrix Analysis and Applications*, 17(1):187–211, 1996.

[23] Zachary Frangella, Joel A. Tropp, and Madeleine Udell. Randomized Nyström preconditioning. *SIAM Journal on Matrix Analysis and Applications*, 44(2):718–752, 2023.

[24] Xi Gao, Jinxin Xiong, Akang Wang, Qihong Duan, Jiang Xue, and Qingjiang Shi. IPM-LSTM: a learning-based interior point method for non-convex optimization. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.

[25] Pieter Ghysels and Wim Vanroose. Hiding global synchronization latency in the preconditioned conjugate gradient algorithm. *Parallel Computing*, 40(7):224–238, 2014.

[26] András Gilyén, Yuan Su, Guang Hao Low, and Nathan Wiebe. Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 193–204, 2019.

[27] Ambros M. Gleixner, Daniel E. Steffy, and Kati Wolter. Iterative refinement for linear programming. *INFORMS Journal on Computing*, 28(3):449–464, 2016.

[28] Jacek Gondzio. Matrix-free interior point method. *Computational Optimization and Applications*, 51:457–480, 2012.

[29] Jacek Gondzio. Convergence analysis of an inexact feasible interior point method for convex quadratic programming. *SIAM Journal on Optimization*, 23(3):1510–1527, 2013.

[30] Jacek Gondzio, Spyridon Pougkakiotis, and John W. Pearson. General-purpose preconditioning for regularized interior point methods. *Computational Optimization and Applications*, 83: 727–757, 2022.

[31] Chen Greif, Erin Moulding, and Dominique Orban. Bounds on eigenvalues of matrices arising from interior-point methods. *SIAM Journal on Optimization*, 24(1):49–83, 2014.

[32] Sander Gribling, Iordanis Kerenidis, and Dániel Szilágyi. An optimal linear-combination-of-unitaries-based quantum linear system solver. *ACM Transactions on Quantum Computing*, 5 (2):9, 2024.

[33] Adrian Harkness, Mohammadhossein Mohammadisiahroudi, Brandon Augustino, Giacomo Nannicini, and Tamás Terlaky. An analysis of iterative refinement for quantum linear system solvers. arXiv:2609.10250, 2026.

[34] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Physical Review Letters*, 103:150502, 2009.

[35] Iordanis Kerenidis and Anupam Prakash. A quantum interior point method for LPs and SDPs. *ACM Transactions on Quantum Computing*, 1(1):5, 2020.

[36] Philip A. Knight, Daniel Ruiz, and Bora Uçar. A symmetry preserving algorithm for matrix scaling. *SIAM Journal on Matrix Analysis and Applications*, 35(3):931–955, 2014.

[37] Lin Lin and Yu Tong. Optimal polynomial based quantum eigenstate filtering with application to solving quantum linear systems. *Quantum*, 4:361, 2020.

[38] Haihao Lu and Jinwen Yang. cuPDLP.jl: A GPU implementation of restarted primal-dual hybrid gradient for linear programming in Julia. *Operations Research*, 73(6):3440–3452, 2025.

[39] Jian Luo, Jie Wang, Hong Wang, Zijie Dong, Zijie Geng, Hanzhu Chen, and Yufei Kuang. Neural Krylov iteration for accelerating linear system solving. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.

[40] István Maros and Csaba Mészáros. A repository of convex quadratic programming problems. *Optimization Methods and Software*, 11(1–4):671–681, 1999.

[41] John M. Martyn, Zane M. Rossi, Andrew K. Tan, and Isaac L. Chuang. Grand unification of quantum algorithms. *PRX Quantum*, 2:040203, 2021.

[42] Mohammadhossein Mohammadisiahroudi, Ramin Fakhimi, and Tamás Terlaky. Efficient use of quantum linear system algorithms in inexact infeasible IPMs for linear optimization. *Journal of Optimization Theory and Applications*, 2024. arXiv:2205.01220.

[43] Mohammadhossein Mohammadisiahroudi, Zeguan Wu, Brandon Augustino, Arielle Carr, and Tamás Terlaky. Improvements to quantum interior point method for linear optimization. *ACM Transactions on Quantum Computing*, 6(1), 2025. arXiv:2310.07574.

[44] Benedetta Morini, Valeria Simoncini, and Mattia Tani. Spectral estimates for unreduced symmetric KKT systems arising from interior point methods. *Numerical Linear Algebra with Applications*, 23(5):776–800, 2016.

[45] Giacomo Nannicini. *Quantum Algorithms for Optimizers*. SIAM, 2025. preprint arXiv:2408.07086.

[46] Gizem Ozbaygin, Burak Kocuk, and Diego A. Moran R. A variational quantum eigensolver-based cutting plane framework for semidefinite programming problems. manuscript, submitted 2026, 2026.

[47] François Pacaud, Sungho Shin, Alexis Montoison, Michel Schanen, and Mihai Anitescu. Condensed-space methods for nonlinear programming on GPUs. *Mathematical Programming Computation*, 2026. arXiv:2405.14236.

[48] Christopher C. Paige and Michael A. Saunders. Solution of sparse indefinite systems of linear equations. *SIAM Journal on Numerical Analysis*, 12(4):617–629, 1975.

[49] Michael L. Parks, Eric de Sturler, Greg Mackey, Duane D. Johnson, and Spandan Maiti. Recycling Krylov subspaces for sequences of linear systems. *SIAM Journal on Scientific Computing*, 28(5):1651–1674, 2006.

[50] Spyridon Pougkakiotis and Jacek Gondzio. An interior point-proximal method of multipliers for convex quadratic programming. *Computational Optimization and Applications*, 78:307–351, 2021.

[51] Shaked Regev, Nai-Yuan Chiang, Eric Darve, Cosmin G. Petra, Michael A. Saunders, Kasia Świrydowicz, and Slaven Peles. HyKKT: a hybrid direct-iterative method for solving KKT linear systems. *Optimization Methods and Software*, 38(2):332–355, 2023.

[52] Daniel Ruiz. A scaling algorithm to equilibrate both rows and columns norms in matrices. Technical Report RAL-TR-2001-034, Rutherford Appleton Laboratory, 2001.

[53] Michel Schubiger, Goran Banjac, and John Lygeros. GPU acceleration of ADMM for large-scale quadratic programming. *Journal of Parallel and Distributed Computing*, 144:55–67, 2020.

[54] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: an operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12:637–672, 2020.

[55] Christoph Sünderhauf, Zalán Németh, Adnaan Walayat, Andrew Patterson, and Bjorn K. Berntson. Matrix inversion polynomials for the quantum singular value transformation. arXiv:2507.15537, 2025.

[56] Ewin Tang. A quantum-inspired classical algorithm for recommendation systems. *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 217–228, 2019.

[57] Yu Tong, Dong An, Nathan Wiebe, and Lin Lin. Fast inversion, preconditioned quantum linear system solvers, fast Green's-function computation, and fast evaluation of matrix functions. *Physical Review A*, 104:032422, 2021.

[58] Joran van Apeldoorn, Arjan Cornelissen, András Gilyén, and Giacomo Nannicini. Quantum tomography using state-preparation unitaries. In *Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1265–1318, 2023.

[59] Shun Wang, Eric de Sturler, and Glaucio H. Paulino. Large-scale topology optimization using preconditioned Krylov subspace methods with recycling. *International Journal for Numerical Methods in Engineering*, 69(12):2441–2468, 2007.

[60] James H. Wilkinson. Rounding errors in algebraic processes. *Prentice-Hall*, 1963.

[61] Margaret H. Wright. Ill-conditioning and computational error in interior methods for nonlinear programming. *SIAM Journal on Optimization*, 9(1):84–111, 1998.

[62] Zeguan Wu, Mohammadhossein Mohammadisiahroudi, Brandon Augustino, Xiu Yang, and Tamás Terlaky. An inexact feasible quantum interior point method for linearly constrained quadratic optimization. *Entropy*, 25(2):330, 2023.

[63] Filippo Zanetti and Jacek Gondzio. A new stopping criterion for Krylov solvers applied in interior point methods. *SIAM Journal on Scientific Computing*, 45(2):A703–A728, 2023.